Tree families for jet classification: accuracy, sparsity and inference cost

29 October 2026

Abstract

We compare three families of shallow hard trees for jet classification: axis-aligned cuts with linear leaves, oblique cuts with constant leaves, and oblique cuts with linear leaves, against a 100-tree XGBoost reference. Tree Alternating Optimization minimizes a shared cross-entropy loss; L1 support selection followed by unpenalized frozen-support refitting reduces coefficient budgets. A ten-model comparison spans smaller and higher-efficiency representatives of each family on fixed data partitions. The highest eligible oblique/linear model reaches 57.73% signal efficiency at 1% background with 1275 nonzero scalars. A sparse three-tree oblique refit reaches 56.20% with 418, while three axis-aligned depth-four trees reach 56.32% with 479 plus discrete cut choices. XGBoost reaches 55.48% with 3099 scalars. The latter two compact candidates reverse their arithmetic ranking between selected-path and all-node evaluation. Matched C++ timings, realizable thresholds and supporting fixed-point checks provide implementation evidence; synthesized FPGA resources and latency remain unmeasured.

1 Introduction

Fast classification under hardware resource constraints requires balancing prediction quality, coefficient storage and arithmetic workload. An axis-aligned tree tests one feature at a node; an oblique tree tests a linear combination. A constant leaf assigns one score to its region, whereas a linear leaf captures first-order feature dependence inside that region. These choices distribute expressivity differently between routing and prediction. Linear leaves in axis-aligned trees have a long history as model trees, including classification applications [1]. Their cost must be assessed at the level of the complete classifier, rather than by comparing isolated nodes or leaves.

We compare three combinations of these choices: axis-aligned cuts with linear leaves, oblique cuts with constant leaves, and oblique cuts with linear leaves. A boosted ensemble of axis-aligned, constant-leaf trees provides the reference. The application distinguishes quark/gluon jets from W, Z and top jets using 16 high-level features, motivated by FPGA inference in the CMS Phase-II Level-1 trigger. The central result is a ten-model comparison of validation performance, learned nonzero scalars, matched C++ evaluation time, and two arithmetic schedules. Its summary table is placed last in both document editions.

Tree Alternating Optimization (TAO) revisits existing cuts instead of fixing them in a single greedy construction [2]; sparse oblique trees have also been used to analyze learned feature representations [3]. Here leaves and cuts are trained against the common loss of an additive forest. We separate support selection from coefficient shrinkage through L1 regularization, support freezing and unpenalized refitting (Select–Freeze–Refit), following the motivation of post-selection methods such as relaxed Lasso [4]. Detailed derivations below define both the oblique proposals and the exhaustive axis-aligned node search. Supporting appendices retain the full refit and constant-leaf studies, numerical feasibility checks and actual learned functions.

2 Tree families and inference cost

2.1 Cuts, paths and leaf functions

An event is represented by a feature vector $x \in \mathbb{R}^F$ and a class label $y \in \{0, \dots, K-1\}$. A forest contains T binary trees of depth D, where depth counts decisions on a full root-to-leaf path. We label a node by its path p from the root: $p = \varnothing$ denotes the root, L its left child, R its right child, and RL the left child of R. Internal paths have length $|p| < D$; leaf paths have length D. Appending L or R identifies the two children. This notation describes the tree independently of storage.

At internal node p of tree $t \in \{1, \dots, T\}$, an affine function

$$h_{tp}(x) = w_{tp}^{\mathsf{T}} x + b_{tp}, \qquad p \longrightarrow \begin{cases} pR, & h_{tp}(x) > 0, \\ pL, & h_{tp}(x) \le 0, \end{cases} \tag{1}$$

determines the next branch. Here $w_{tp} \in \mathbb{R}^F$ is the cut's feature weight vector and $b_{tp} \in \mathbb{R}$ its bias. Routing is hard: each event follows exactly one branch. A cut is axis-aligned when only one component of w_{tp} is nonzero, and oblique when it combines several features.

For the axis-aligned family we fix the normal's magnitude and use

$$w_{tp} = s_{tp} e_{j_{tp}}, \qquad h_{tp}(x) = s_{tp} x_{j_{tp}} + b_{tp}, \qquad j_{tp} \in \{1, \dots, F\}, \quad s_{tp} \in \{-1, +1\}, \tag{2}$$

where e_j is a coordinate unit vector. A cut therefore learns a discrete feature index, a direction and one scalar threshold; it needs no general feature multiplication. Input-independent cuts remain allowed. Together with Eq. (3), this defines the three families compared below. Oblique cuts retain unconstrained multivariate normals, including axis-aligned cases.

A complete depth-D tree has $I = 2^D - 1$ internal nodes and $L_D = 2^D$ leaves. Its effective topology can be smaller. If $w_{tp} = 0$, the bias determines a constant route, and the other child and its descendants are unreachable for *every* input. For example, $w_{tp} = 0, b_{tp} = 0$ always chooses the left child by Eq. (1). Such branches can be removed without changing the function. A branch merely unused by a finite data sample is not necessarily unreachable.

Let $\ell_t(x)$ be the leaf path reached in tree t. With class $K-1$ as a reference of zero logit, a linear leaf stores $K-1$ affine functions

$$f_{t\ell k}(x) = a_{t\ell k}^{\mathsf{T}} x + c_{t\ell k}, \qquad k = 0, \dots, K-2, \tag{3}$$

where $a_{t\ell k} \in \mathbb{R}^F$ and $c_{t\ell k} \in \mathbb{R}$ are the leaf weights and biases. A constant leaf restricts $a_{t\ell k} = 0$. The forest logits and probabilities are

$$z_k(x) = \sum_{t=1}^{T} f_{t,\ell_t(x),k}(x), \quad z_{K-1}(x) = 0, \qquad P(y = k \mid x) = \frac{\exp z_k(x)}{\sum_{q=0}^{K-1} \exp z_q(x)}. \tag{4}$$

Thus tree logits are added before applying softmax. Linear leaves allow first-order dependence within a region, but do not enforce continuity across its boundaries. The forest is affine on each intersection of its trees' leaf regions; adding trees refines this partition.

For the binary application, class 0 is background and class 1 is signal. The signal margin and probability are

$$m(x) = -z_0(x), \qquad P(y = 1 \mid x) = \frac{1}{1 + \exp[-m(x)]}. \tag{5}$$

A larger margin therefore corresponds to a more signal-like event.

2.2 Parameter and arithmetic counts

Counting scalar weights and biases gives

$$P_{\text{dense}} = T\big[(2^D - 1)(F + 1) + 2^D B(K - 1)\big], \qquad B = \begin{cases} F + 1, & \text{linear leaves,} \\ 1, & \text{constant leaves.} \end{cases} \tag{6}$$

This is the allocated coefficient count for a complete topology. We separately report P_{nz}, the number of coefficients exactly different from zero, and P_{reach}, the nonzero count after removing algebraically unreachable branches. These counts include biases and exclude feature indices, connectivity and preprocessing metadata.

For an axis-aligned tree, the fixed unit normal is not a learned scalar vector. Its continuous allocation is instead

$$P_{\text{cont}}^{\text{axis}} = T\big[(2^D - 1) + 2^D(F + 1)(K - 1)\big] \tag{7}$$

for linear leaves, plus one feature index and direction bit per nonconstant cut. The reported learned nonzero count includes active scalar thresholds and leaf coefficients, not fixed unit weights or constant-routing flags. For axis/linear layouts 1×D3, 1×D4, 3×D3 and 3×D4, Eq. (7) gives 143, 287, 429 and 861 scalar slots. Sparse representations also need indices; nonzero scalar count is not a complete memory-footprint estimate. All main comparison counts use standardized inputs.

For $F = 16, K = 2, D = 4$, one linear-leaf tree has $15 \cdot 17 + 16 \cdot 17 = 527$ coefficient slots, while three have 1,581. A single dense tree may therefore have a similar *nonzero* count to a more strongly regularized forest of three trees, despite its smaller allocated count.

A dense binary forest with linear leaves needs at most $TF(D + 1)$ feature multiplications and TD comparisons along a selected path. Bias additions are not multiplications. Evaluating all internal functions and leaves instead uses $TF(2^{D+1} - 1)$ multiplications. Sparse functions reduce these counts according to their retained feature weights. They are arithmetic bounds; latency also depends on parallelism, multiplier reuse, precision and memory.

For the final comparison we also remove exact $+1$ and -1 factors, implemented by selection or sign change. Define $c(g) = \#\{f : u_f \notin \{0, -1, +1\}\}$ for an affine function $g(x) = u^\top x + d$. Let $\mathcal{L}_t^r$ and $\mathcal{I}_t^r$ be the leaves and internal paths remaining after input-independent route pruning. For free class logits, the two multiplication counts are

$$M_{\text{path}} = \sum_t \max_{\ell \in \mathcal{L}_t^r} \left[\sum_{p \prec \ell} c(h_{tp}) + \sum_{k=0}^{K-2} c(f_{t\ell k}) \right], \tag{8}$$

$$M_{\text{all}} = \sum_t \left[\sum_{p \in \mathcal{I}_t^r} c(h_{tp}) + \sum_{\ell \in \mathcal{L}_t^r} \sum_{k=0}^{K-2} c(f_{t\ell k}) \right]. \tag{9}$$

Here $p \prec \ell$ denotes a retained internal ancestor of leaf ℓ. The first count is an upper bound for sequential selected-path evaluation; the second evaluates every reachable affine function, as in a parallel implementation. Maximal paths in different trees need not be simultaneously realizable. Bias additions, comparisons and score aggregation are excluded. Other constant-multiplier optimizations, including shifts, are not assumed. Rescaling a one-feature cut into a pure comparison could further reduce work, but requires its own numerical routing checks and is not assumed here. These operation counts cannot be equated with DSP usage or FPGA latency.

Implementation and reproducibility

Storage and inference. `HardTree` and `HardTAOClassifier` in `src/tao/hard.py` implement Eqs. (1)–(4). Internal nodes use heap indices $j = 0, \ldots, I - 1$, with children $2j + 1, 2j + 2$; leaf-array indices subtract I from the heap index. The arrays are `split_w`, `split_b` and `leaf_parameters`. The methods `routes`, `leaf_indices`, `predict_subtree`, `decision_function`

and `predict_proba` implement traversal and logits. Input and logit arithmetic is binary64; public probabilities are cast to float32. The native ROC uses those probabilities, including any ties. Portable margin inference remains binary64. The current portable schema is binary, although the fitting code supports multiclass logits.
`sparsity_profile` in `src/tao/sparsity.py` computes exact-zero and reachable counts. The existing NumPy `dense` traversal may compute a node function over an entire batch before selecting reaching rows; its runtime does not represent a generated sparse path kernel.

3 Tree Alternating Optimization

3.1 A common forest objective

For training events (x_i, y_i), $i = 1, \dots, N$, let $v_i \geq 0$ be event weights and $W = \sum_i v_i > 0$. All results below use $v_i = 1$. We minimize

$$J(\theta) = \frac{1}{W} \sum_{i=1}^{N} v_i \, \mathrm{CE}(y_i, z(x_i)) + \lambda_s \sum_{t,p} \left(\|w_{tp}\|_1 + |b_{tp}| \right) + \lambda_\ell \sum_{t,\ell,k} \|a_{t\ell k}\|_1 , \qquad \mathrm{CE}(y, z) = \log \sum_{k=0}^{K-1} e^{z_k} - z_y. \tag{10}$$

Here θ denotes all cut and leaf parameters, $\lambda_s \geq 0$ penalizes cuts and $\lambda_\ell \geq 0$ penalizes leaf feature weights. Leaf biases are unpenalized; cut biases are penalized in oblique training. Axis training fixes unit normals and sets $\lambda_s = 0$; its feature and direction choices are discrete. For constant leaves, λ_ℓ has no effect. Every subproblem retains the full training-weight denominator W, so a penalty has the same normalization at different depths and node occupancies.

TAO alternates optimization blocks while other parameters are fixed [2]. An *epoch* visits every tree, updates its leaves and then its internal nodes from the bottom up, and finally refits all forest leaves jointly. Other trees contribute fixed logit offsets during each individual-tree update. Their offsets are refreshed before updating the next tree. Earlier trees remain trainable; the procedure does not freeze them as later trees are added.

3.2 Leaf optimization

Fix every cut, so the leaf reached by each training event is fixed. For leaf ℓ of tree t, define its event set $\mathcal{I}_{t\ell} = \{i : \ell_t(x_i) = \ell\}$. Introduce the leaf basis $\phi(x) = (x_1, \dots, x_F, 1)^\mathsf{T} \in \mathbb{R}^B$ for linear leaves and $\phi(x) = 1$ for constant leaves. Let $\beta_{t\ell k}$ collect the coefficients of that basis: it is $(a_{t\ell k}, c_{t\ell k})$ for a linear leaf and just $c_{t\ell k}$ for a constant leaf. For $i \in \mathcal{I}_{t\ell}$,

$$z_{ik} = o_{ik} + \phi(x_i)^\mathsf{T} \beta_{t\ell k}, \qquad o_{ik} = \sum_{s \neq t} f_{s,\ell_s(x_i),k}(x_i), \qquad k < K - 1, \tag{11}$$

with $z_{i,K-1} = o_{i,K-1} = 0$. Thus the other trees enter as a fixed offset, not as an additional target to be fitted.

Terms in Eq. (10) from other leaves and all cuts are constant in this update. Removing them leaves the subproblem

$$J_{t\ell}(\beta) = \frac{1}{W} \sum_{i \in \mathcal{I}_{t\ell}} v_i \, \mathrm{CE}(y_i, z_i(\beta)) + \lambda_\ell \sum_{b=1}^{B} \sum_{k=0}^{K-2} \rho_b |\beta_{t\ell k,b}|, \tag{12}$$

where $\rho = (1, \dots, 1, 0)$ for a linear leaf and $\rho = 0$ for a constant leaf. In particular, the bias is unpenalized and the denominator remains W, not the weight sum in $\mathcal{I}_{t\ell}$.

Write $\pi_{ik} = \exp(z_{ik}) / \sum_{j=0}^{K-1} \exp(z_{ij})$ for the current class probabilities, and let $\mathbf{1}[A]$ equal 1 when statement A holds and 0 otherwise. Differentiating the cross-entropy gives

$$\frac{\partial \operatorname{CE}(y_i, z_i)}{\partial z_{ik}} = \pi_{ik} - \mathbf{1}[y_i = k], \qquad \frac{\partial L_{t\ell}}{\partial \beta_{t\ell k,b}} = \frac{1}{W} \sum_{i \in \mathcal{I}_{t\ell}} v_i \phi_b(x_i)(\pi_{ik} - \mathbf{1}[y_i = k]), \tag{13}$$

where $L_{t\ell}$ denotes the unpenalized term in Eq. (12). Different leaves of this tree involve disjoint event sets, so they can be fitted separately while its other-tree offset is held fixed.

The joint forest leaf refit has the same form, but includes every tree in one design matrix. Index its columns by (t, ℓ, b) and define

$$A_{i,(t,\ell,b)} = \mathbf{1}[\ell_t(x_i) = \ell]\, \phi_b(x_i), \qquad \Theta_{(t,\ell,b),k} = \beta_{t\ell k,b}. \tag{14}$$

Here $A \in \mathbb{R}^{N \times (TL_D B)}$ and $\Theta \in \mathbb{R}^{(TL_D B) \times (K-1)}$. Thus $z_{ik} = (A\Theta)_{ik}$ for $k < K-1$, with the reference logit zero. If $q = (t, \ell, b)$ indexes a design column and $\rho_q = \rho_b$, its objective and smooth gradient are

$$J_{\text{leaf}}(\Theta) = \underbrace{\frac{1}{W} \sum_i v_i \operatorname{CE}(y_i, z_i(\Theta))}_{L(\Theta)} + \lambda_\ell \sum_{q,k} \rho_q |\Theta_{qk}|, \tag{15}$$

$$\nabla_\Theta L = \frac{1}{W} A^\top R, \qquad R_{ik} = v_i(\pi_{ik} - \mathbf{1}[y_i = k]), \quad k < K-1. \tag{16}$$

Cross-entropy is convex in its logits, the logits are affine in the leaf coefficients, and the L1 term is convex. Both the individual-leaf and joint problems are therefore convex. The joint parameters need not be unique: different trees can contribute compensating logits.

To handle L1 with a smooth bound-constrained solver, flatten the coefficients being fitted into scalars ϑ_r, and let λ_r be their penalty strength ($\lambda_\ell \rho_b$ for a leaf coordinate). For each penalized coordinate, write $\vartheta_r = u_r^+ - u_r^-$ with $u_r^+, u_r^- \geq 0$. This is an exact reformulation because

$$\min_{u^+, u^- \geq 0:\ u^+ - u^- = \vartheta} \lambda_r(u^+ + u^-) = \lambda_r |\vartheta|. \tag{17}$$

Coordinates with $\lambda_r = 0$ are optimized directly as unconstrained real variables. For either smooth leaf loss L, the auxiliary objective and its penalized-coordinate derivatives are

$$\widetilde{J} = L(\vartheta(u)) + \sum_{r:\lambda_r > 0} \lambda_r(u_r^+ + u_r^-), \qquad \frac{\partial \widetilde{J}}{\partial u_r^+} = g_r + \lambda_r, \quad \frac{\partial \widetilde{J}}{\partial u_r^-} = -g_r + \lambda_r, \tag{18}$$

where $g_r = \partial L / \partial \vartheta_r$ is obtained from Eq. (13) or (16). The free-coordinate derivative remains g_r. We minimize this differentiable objective using a bound-constrained quasi-Newton solver and reconstruct the stored coefficients as $\vartheta_r = u_r^+ - u_r^-$.

At a finite iteration budget, both parts can remain positive. Define the actual coefficient objective as $J_{\text{actual}} = L(\vartheta) + \sum_r \lambda_r |\vartheta_r|$. The auxiliary objective exceeds it by the explicit gap

$$\widetilde{J} - J_{\text{actual}} = \sum_{r:\lambda_r > 0} \lambda_r (u_r^+ + u_r^- - |u_r^+ - u_r^-|) = 2 \sum_{r:\lambda_r > 0} \lambda_r \min(u_r^+, u_r^-) \geq 0. \tag{19}$$

Canonical parts $\max(\vartheta_r, 0)$ and $\max(-\vartheta_r, 0)$ remove this gap without changing any logit. We evaluate the candidate with the actual L1 norm and accept it only if its leaf objective does not increase. Since the omitted terms are fixed, this also ensures nonincrease of the full objective in Eq. (10).

3.3 Decision-node optimization

Let $\mathcal{R}_{tp}$ be the training events reaching node p in tree t. Nodes with no reaching events are left unchanged; the following update assumes $\mathcal{R}_{tp} \neq \varnothing$. Keep its ancestors, descendants, leaf functions and all other trees fixed. Ancestors determine $\mathcal{R}_{tp}$, so changing this cut cannot affect any event outside that set. For $i \in \mathcal{R}_{tp}$, let $g_L(x_i)$ and $g_R(x_i)$ be the fixed logit vectors predicted by its two subtrees, and o_i the logit sum from the other trees. The two possible losses are

$$C_i^L = \mathrm{CE}(y_i, o_i + g_L(x_i)), \qquad C_i^R = \mathrm{CE}(y_i, o_i + g_R(x_i)), \qquad \Delta_i = C_i^L - C_i^R. \tag{20}$$

Write $r_i(w,b) = \mathbf{1}[w^\mathsf{T} x_i + b > 0]$ for the proposed right-branch decision. All other penalties and all outside-event losses are constant, so Eq. (10) reduces to

$$Q(w,b) = \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i \big[(1 - r_i(w,b)) C_i^L + r_i(w,b) C_i^R\big] + \lambda_s(\|w\|_1 + |b|). \tag{21}$$

Define a preferred-branch label $q_i = \mathbf{1}[\Delta_i > 0]$ and an effective weight $\gamma_i = v_i|\Delta_i|$. For either decision $r \in \{0,1\}$,

$$(1-r)C_i^L + rC_i^R = \min(C_i^L, C_i^R) + |\Delta_i|\,\mathbf{1}[r \neq q_i]. \tag{22}$$

Subtracting the first, cut-independent term gives the equivalent weighted misrouting objective

$$Q(w,b) - \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i \min(C_i^L, C_i^R) = \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} \gamma_i \mathbf{1}[r_i(w,b) \neq q_i] + \lambda_s(\|w\|_1 + |b|). \tag{23}$$

Thus the reduced labels describe preferred branches, even when the original forest classifies more than two classes. A large $|\Delta_i|$ makes a wrong branch expensive. Events with $\gamma_i = 0$ are indifferent or have zero training weight and give no information for this cut.

Logistic proposal. The discrete misrouting term has no useful ordinary gradient in (w,b). When both branch labels occur among active events, we form a smooth proposal by assigning the right branch probability $\sigma(h_i)$, where $h_i = w^\mathsf{T} x_i + b$ and $\sigma(h) = 1/(1+e^{-h})$. The Bernoulli negative log-likelihood for branch label q_i is

$$-q_i \log \sigma(h_i) - (1-q_i)\log[1-\sigma(h_i)] = \log(1+e^{h_i}) - q_i h_i. \tag{24}$$

This gives the weighted proposal objective

$$S(w,b) = \underbrace{\frac{1}{W} \sum_{i \in \mathcal{R}_{tp}:\gamma_i > 0} \gamma_i \big[\log(1+e^{h_i}) - q_i h_i\big]}_{L_{\log}(w,b)} + \lambda_s(\|w\|_1 + |b|), \tag{25}$$

with smooth derivatives

$$\nabla_w L_{\log} = \frac{1}{W} \sum_i \gamma_i [\sigma(h_i) - q_i] x_i, \qquad \frac{\partial L_{\log}}{\partial b} = \frac{1}{W} \sum_i \gamma_i [\sigma(h_i) - q_i], \tag{26}$$

where these sums run over the same active events as Eq. (25). The L1 penalty includes the bias and can be handled by the positive/negative parameterization above. Let $(\widehat{w}, \widehat{b})$ denote the resulting approximate minimizer. A decrease in S does not imply a decrease in the hard loss Q, so the proposal must be checked against Eq. (21).

Threshold search and acceptance. Let $(w^{(0)}, b^{(0)})$ be the current cut. We consider directions

$$\mathcal{D} = \{w^{(0)}, \widehat{w}, 0\} \cup \{(1-\eta)w^{(0)} + \eta\widehat{w} : \eta \in \{0.25, 0.5, 0.75\}\}. \tag{27}$$

For a fixed direction $d \in \mathcal{D}$, set $s_i = d^\mathsf{T} x_i$ and $\tau = -b$, so right routing means $s_i > \tau$. Let $a_1 < \cdots < a_M$ be the distinct projected scores of events in $\mathcal{R}_{tp}$. The intervals

$$I_0 = (-\infty, a_1), \qquad I_m = [a_m, a_{m+1}) \ (1 \le m < M), \qquad I_M = [a_M, \infty) \tag{28}$$

exhaust the possible routings along d. The half-open convention follows from sending equality left. Equal scores move together; the endpoints include all-right and all-left routing.

To evaluate every interval, start with all events on the right and define

$$U = \sum_{i \in \mathcal{R}_{tp}} v_i C_i^R, \qquad B_m = \sum_{i \in \mathcal{R}_{tp} : s_i \le a_m} v_i (C_i^L - C_i^R), \qquad B_0 = 0. \tag{29}$$

The values B_m are obtained by sorting the scores once and accumulating whole tied groups. For $\tau \in I_m$, the objective is

$$Q(d, -\tau) = \frac{U + B_m}{W} + \lambda_s \|d\|_1 + \lambda_s |\tau|. \tag{30}$$

Only the bias penalty varies within an interval. We therefore choose its representable finite threshold nearest zero,

$$\tau_m^* \in \operatorname*{arg\,min}_{\tau \in I_m \cap \mathbb{F}} |\tau|, \tag{31}$$

where $\mathbb{F}$ is the set of finite numbers in the working floating-point format; empty intersections are discarded. This is zero if zero belongs to the interval, the included lower endpoint for an entirely positive interval, and the representable predecessor of the open upper endpoint for an entirely negative interval. The two outer intervals use the same rule. When $\lambda_s = 0$, every threshold in an interval has the same loss; we keep this representative convention.

For each direction d, let $m(d)$ minimize Eq. (30) over its nonempty intervals. We compare the resulting pairs $(d, -\tau^*_{m(d)})$, the raw logistic pair $(\widehat{w}, \widehat{b})$, and the unchanged cut. Each of these proposed cuts is reevaluated using actual hard routing, including both weight and bias penalties. We update only when the best candidate satisfies

$$Q(w^{\text{new}}, b^{\text{new}}) < Q(w^{(0)}, b^{(0)}). \tag{32}$$

Because all omitted terms are fixed, this strictly decreases the full forest objective. The threshold search is exhaustive for each proposed direction; the finite direction set does not exhaust all hyperplanes. In fixed-support refitting, directions must preserve frozen-zero weights, and a frozen-zero bias permits only $\tau = 0$ rather than unrestricted threshold search.

A lower-bound certificate and one-sided labels. During unpenalized fixed-support refitting, $\lambda_s = 0$ and Eq. (23) gives the bound

$$Q(w, b) \ge Q_{\text{lb}} := \frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i \min(C_i^L, C_i^R), \qquad Q(w, b) - Q_{\text{lb}} = \frac{1}{W} \sum_i \gamma_i \mathbf{1}[r_i(w, b) \ne q_i]. \tag{33}$$

The last sum is over $\mathcal{R}_{tp}$. If the current cut chooses the preferred child for every event with $\gamma_i > 0$, the gap is zero. It already minimizes this node's hard subproblem, even over all feasible cuts, so an auxiliary logistic fit is unnecessary. This is a local certificate with subtrees and other trees fixed, not a statement about the full forest optimum.

When active events contain only one branch label, we skip the logistic fit and retain only the current and zero directions in the threshold search. The zero direction gives a constant route whenever the allowed bias permits it. If all $\gamma_i = 0$, any route has the same data loss; the current cut already attains the unpenalized lower bound. These cases introduce no third branch and do not require a finite optimum of a one-class logistic problem.

3.4 Exhaustive axis-aligned node updates

For Eq. (2), keep the same reaching set and child losses of Eq. (20). Set $w = se_j$, $u = -b$ and $\lambda_s = 0$ in Eq. (21). Then

$$Q_{j,s}(u) = \frac{1}{W}\left[\sum_{i:sx_{ij}\le u} v_i C_i^L + \sum_{i:sx_{ij}>u} v_i C_i^R\right], \quad i \in \mathcal{R}_{tp}. \tag{34}$$

For each of the $2F$ pairs (j, s), sort sx_{ij}, move tied groups together, and use the prefix sums of Eqs. (29)–(30) with zero cut penalty. Every distinct routing interval, including both constant endpoints, is considered. An input-independent zero direction is also included. The minimizing candidate is reevaluated with the actual hard decision; an update is accepted only if Eq. (32) holds. The current cut is retained on a tie. If all effective weights vanish, it is already optimal for the fixed child losses and remains unchanged.

This search exhausts the axis-aligned coordinate subproblem, with other cuts, leaves and trees fixed. It does not establish a global optimum of the forest. It uses the current linear leaf functions when scoring each split, rather than first learning a constant-leaf partition and only then replacing its leaves. During the axis cohort's unpenalized refit, cuts and exact-zero leaf support are frozen, so only the allowed leaf coefficients are optimized.

Implementation and reproducibility

`AxisTAOClassifier` in `src/tao/axis.py` overrides only node proposal search; its leaf solvers and resumable epoch follow the common implementation. `assert_axis` enforces zero or signed-unit one-feature normals; `axis_parameter_counts` separates continuous values and discrete choices. `scripts/train_axis_tao.py` isolates this experiment from existing oblique runs and checks pinned source hashes. Exhaustive small binary/multiclass tests, equality routing, tied losses, checkpoint/resume identity and frozen leaf refitting are covered by `tests/test_axis_tao.py`.

3.5 Initialization and numerical stopping

Fresh trees are initialized by greedy axis-aligned classification trees fitted to bootstrap draws from the training sample. Subsequent TAO updates use the full training sample. Leaf class counts receive a 0.5 pseudocount before conversion to reference-class log odds; each tree's initial logits are divided by T. Early terminal nodes can be embedded in a complete topology using constant left-routing cuts, which explains some unreachable branches. The sparsity study below continues an existing trained forest.

We distinguish an optimizer iteration, an epoch of alternating tree updates, and convergence of the full run. Reaching an iteration cap alone is not convergence. For fixed routes, let g_q be the gradient of normalized unpenalized cross-entropy with respect to leaf coordinate θ_q, and let λ_q equal λ_ℓ for a penalized feature weight and zero for a bias. The leaf stationarity residual is

$$r_q = \begin{cases} 0, & q \text{ frozen}, \\ |g_q + \lambda_q \operatorname{sign}(\theta_q)|, & q \text{ free}, \ |\theta_q| > \delta, \\ \max(|g_q| - \lambda_q, 0), & q \text{ free}, \ |\theta_q| \le \delta, \end{cases} \qquad r_{\mathrm{KKT}} = \max_q r_q, \tag{35}$$

with numerical zero tolerance $\delta = 10^{-10}$. Here KKT denotes the Karush–Kuhn–Tucker stationarity conditions. Frozen coordinates are subject to equality constraints and do not require zero gradients. The tolerance δ is used only for this residual; support selection uses exact zeros.

We require $r_{\mathrm{KKT}} \le 10^{-6}$, successful inner solves, relative objective gain at most 10^{-5} and a routing-change fraction at most 10^{-4} for three consecutive epochs, after at least five epochs. An event counts as rerouted if its leaf changes in any forest member. The relative gain is

$\max(0, (J_{\text{before}} - J_{\text{after}})/\max(|J_{\text{before}}|, 10^{-12}))$; increases above 10^{-8} are rejected. These conditions concern numerical stationarity of leaf fits and the available cut proposals. They do not establish global optimality of the forest.

Implementation and reproducibility

Objective and leaf solvers. `src/tao/hard.py` contains `cross_entropy_rows`, `ce_value_gradient`, `fit_leaf_parameters`, `HardTAOClassifier._penalty`, `objective`, `_joint_design` and `global_refit`. The joint CSR design activates a leaf block in each tree: a linear-leaf row has $T(F+1)$ stored entries. For 531,200 events and three trees this is 27,091,200 entries. Frozen feature columns are still stored. The solver is SciPy L-BFGS-B. `fit_l1_split` also supports scikit-learn LIBLINEAR for ordinary penalized proposals, with $C = 1/(\lambda_s W)$ and a penalized synthetic intercept. Zero-penalty and masked proposals use L-BFGS-B.
Cuts and epochs. `best_threshold` in `src/tao/split_search.py` implements the threshold search; `FrozenSupportTAO._split_update` in `src/tao/support.py` handles fixed support and pointwise certificates. `begin_epoch`/`step_epoch` in `src/tao/epoch.py` serialize a bottom-up epoch and cache other-tree logits. `initialize_cart` in `src/tao/initialization.py` initializes fresh trees. `append_zero_tree` can extend a forest while preserving predictions; the selected compact forest was not obtained with that option. `leaf_kkt_residual` and `convergence_check` in `src/tao/convergence.py` implement Eq. (35) and the stopping tests.

4 Selecting support and refitting

4.1 Select–Freeze–Refit

L1 regularization both removes coefficients and shrinks surviving values. Once a coefficient will be retained, shrinkage can unnecessarily reduce accuracy. Motivated by post-selection refitting [4], we use the following procedure, termed Select–Freeze–Refit (SFR):

1. **Select.** Follow split-only, leaf-only or joint L1 paths using converged terminal fits as warm starts. Choose a support strategy on validation.
2. **Freeze.** Copy the selected parent, fix its topology, and record $M_q = \mathbf{1}[\theta_q \neq 0]$ for every coefficient, including biases.
3. **Refit.** Remove both L1 penalties. First optimize the surviving leaf coefficients with cuts fixed; then optionally alternate cut and leaf updates while preserving the original masks.

Let $\mathcal{C}$ denote coordinates of input-independent cuts and $\theta_{\mathcal{C}}^{\text{parent}}$ their parent values. The full refit minimizes

$$\frac{1}{W}\sum_i v_i \,\mathrm{CE}(y_i, z(x_i;\theta)) \quad \text{subject to } \theta_q = 0 \text{ if } M_q = 0, \quad \theta_{\mathcal{C}} = \theta_{\mathcal{C}}^{\text{parent}}. \tag{36}$$

Free coefficients may change sign or pass through zero; their masks are not reselected. Frozen-zero coefficients cannot regrow. Cuts that were already input-independent retain their parameters and constant decision. A frozen-zero cut bias also remains zero during threshold search. This differs from simply lowering L1 on an unrestricted forest, which could restore removed parameters.

Each stage begins with a joint fixed-route leaf fit; the full stage repeats it after each epoch of cut updates. Model selection uses the declared validation working point among eligible saved fits. A run that eventually converges can have its best validation score at an earlier epoch; the selected snapshot need not itself be stationary.

4.2 Separation and cut-scale ambiguity

Unpenalized logistic fits can exhibit complete or quasi separation, with the loss approaching an infimum as coefficients diverge. With routes fixed, logits are linear in free leaf coordinates. We search for a free direction d such that every true-class margin against another class has a nonnegative change and at least one has a positive change:

$$\frac{d}{d\alpha}\left[z_{y_i}(x_i;\theta+\alpha d)-z_k(x_i;\theta+\alpha d)\right]\geq 0 \quad (v_i>0,\ k\neq y_i). \tag{37}$$

These are linear inequalities in d. A box-constrained linear program fixes the scale of the direction and searches for a positive summed improvement. A positive feasible result indicates a separating direction; an ambiguous result or timeout is held as inconclusive. Such stages are excluded from selection, while a separately completed leaf-only stage can remain eligible. This diagnostic does not establish uniqueness of the fitted coefficients.

Cut scale presents a separate issue. For any $c>0$, in exact arithmetic,

$$\mathbf{1}[w^{\mathsf{T}}x+b>0]=\mathbf{1}[cw^{\mathsf{T}}x+cb>0], \qquad \|cw\|_1+|cb|=c(\|w\|_1+|b|). \tag{38}$$

The hard-routing loss is unchanged by positive rescaling, while raw-coefficient L1 decreases as c decreases. The logistic proposal fixes a scale within its smooth subproblem, but the outer hard objective remains scale-ambiguous. Thus L1 is an empirical sparsity control in this alternating procedure, not an invariant measure of tree size. Small nonzero cut coefficients cannot simply be discarded: their ratios may still define an important boundary. Explicit rescaling before quantization requires verification near boundaries. An input-independent cut can instead be replaced by its Boolean branch choice.

Implementation and reproducibility

Masks and separation. `freeze_support`, `continue_full_refit`, `FrozenSupportTAO` and `separation_check` are in `src/tao/support.py`. They check topology, shapes, finite values, mask hashes and absence of zero regrowth. Leaf-only cuts remain bit-identical. The separation LP uses all positive-weight training events, checks feasibility, and treats scores above 10^{-7} as positive and at or below 10^{-10} as near-zero; intermediate values and 600-second timeouts are held. Certificates are cached only while routes remain unchanged.
A concrete precision trap occurs in the compact model: scientific node $(t,p)=(2,RR)$ (storage tree 1, node 6) has zero weights and bias $2^{-1074}>0$. It always chooses right. Rounding the bias to zero changes the decision; folding the constant branch preserves it without representing a subnormal constant in fixed-point arithmetic.

5 Jet-classification study

5.1 Data and comparison protocol

We use 16 high-level jet features from the dataset catalogued as OpenML 42468 [5]. Class 0 combines quark and gluon jets; class 1 combines W, Z and top jets. The fixed partitions contain 531,200 training, 132,800 validation and 166,000 test events, using split seed 42. Validation contains 52,511 background and 80,289 signal events. Each feature is standardized by its training mean μ_f and standard deviation $s_f>0$:

$$x_f=(r_f-\mu_f)/s_f, \qquad f=1,\dots,F, \tag{39}$$

where r_f denotes the unstandardized feature. All models use the same partitions, labels and feature order.

The primary metric is signal efficiency ϵ_s at background efficiency $\epsilon_b = 0.01$, interpolated on the empirical receiver operating characteristic (ROC). AUC denotes its area. The final comparison uses the increasing signal margin; its sigmoid is monotonic in exact arithmetic, but finite-precision probabilities can introduce additional ties. Saved training checkpoints are selected using the shared probability-based metric and rechecked during export. For a threshold τ, the operating efficiencies are the fractions of signal and background events accepted by a score above τ. Interpolation can give a point not exactly realized by a strict threshold. With the validation sample here, a realizable threshold accepting at most 1% background permits at most $\lfloor 0.01 \cdot 52{,}511 \rfloor = 525$ background events.

All tree-family scans, SFR choices and results below use validation, with AUC secondary to the low-background working point. The test partition was used in an earlier XGBoost reproduction check, but not to select the oblique models or penalties, including the later axis/linear study. Reported uncertainties consequently concern validation and condition on the selected fitted models.

The XGBoost reference is an additive ensemble of axis-aligned trees with constant leaves, trained using gradient/Hessian information for greedy splits and regularized leaf updates [6]. It has 100 trees, maximum depth 4, learning rate 0.1, row and feature subsampling 0.8, histogram-based splits, binary logistic objective and random seed 42. Its realized structure contains 1,499 thresholds, 1,599 leaf values and one base bias: 3,099 scalar parameters. Feature indices and topology are excluded, consistently with the oblique count.

Its increasing signal margin is

$$m_{\mathrm{XGB}}(x) = m_0 + \sum_{t=1}^{100} u_{t,\ell_t(x)}, \qquad P_{\mathrm{XGB}}(y = 1 \mid x) = \sigma(m_{\mathrm{XGB}}(x)), \tag{40}$$

where m_0 is the initial signal log-odds and $u_{t\ell}$ is the scalar leaf contribution with the learning rate already absorbed. Its native cut sends $x_j < \tau_{tp}$ left and equality right, unlike Eq. (1). Early terminal leaves need not have length-four paths. The numerical checks preserve these routing and score conventions.

Implementation and reproducibility

The project repository [7] contains the archived data contract and training configuration `configs/bdt_qg_vs_wzt_seed42.yaml`; `src/train_bdt.py` trains the reference. The input files are `data/qg_vs_wzt_standardized.npz` and `data/qg_vs_wzt_metadata.json`. `scripts/audit_tao_splits.py` checks split identities. The data hash is

`f5ec6f7ed442010656a904700f8f6421cc34b674a1ec810c5cf6b2de003ed926`

No new split is drawn from OpenML. The root README documents the historical XGBoost test reproduction. `src/evaluate.py` supplies the common ROC interpolation. XGBoost version 3.2.0 is used with one thread; remaining settings are those of that version. The handed-over native JSON fixes the actual model. The paper input generator evaluates validation only.

5.2 Training cohorts and representative selection

The comparison contains three trainable tree families: axis-aligned cuts with linear leaves, oblique cuts with constant leaves, and oblique cuts with linear leaves. Their common reference is the 100-tree axis-aligned, constant-leaf boosted ensemble. Table 8, placed last in the document, contains the ten selected configurations. Within each family we retain its highest eligible working-point efficiency and two smaller representatives, rather than the three largest efficiencies irrespective of size. This is a compact selection of operating points, not an exhaustive Pareto frontier.

The axis/linear cohort uses $T \in \{1, 3\}$ and $D \in \{3, 4\}$, giving four layouts, with $\lambda_s = 0$ and

$\lambda_\ell = 10^{-5}$. All four source runs converged. Each terminal source then supplied its exact leaf support for an unpenalized leaf-only refit, with cuts held fixed. Three refits converged; the three-tree depth-four refit detected logistic separation and was held. Its regularized source remains eligible. For each layout we select the best saved working-point checkpoint from eligible source/refit stages. The selected three-tree depth-three model has 274 learned nonzero scalars and 53.83% efficiency; the single depth-four tree has 228 and 54.74%. We therefore omit the former from the three representatives, although its AUC is slightly higher. This decision follows the stated low-background objective. All four layouts are reported in Appendix B.

The oblique/constant cohort scans $T \in \{1, 3, 5\}$, $D \in \{2, 3, 4\}$ and $\lambda_s \in \{0, 10^{-7}, 10^{-5}\}$: all 27 source configurations converged. The nominal leaf penalty has no effect because constant leaves have no feature weights. Selecting the best eligible penalty/checkpoint for each layout gives the three depth-four representatives with one, three and five trees. The full nine-layout summary is in Appendix C.

For oblique/linear forests, the architecture study varies tree count, depth and separate cut/leaf penalties, while a further support study applies Select–Freeze–Refit. At the comparison snapshot, 57 of 58 architecture configurations had converged; unfinished sources are excluded. The highest eligible efficiency is 57.73% from five depth-four trees, trained with $\lambda_s = \lambda_\ell = 10^{-5}$ and selected at epoch 21 of its converged source run. The other representatives are a completed leaf-only refit with 837 nonzeros at 57.12%, and the 418-nonzero Compact refit at 56.20%. The former's subsequent full refit was held and is excluded. Compact refit's full source stage converged at epoch 15; its selected snapshot is epoch 8. The support study and training chain are given in Appendix A.

Convergence is required of the source stage, not necessarily of an earlier validation-selected snapshot. Different families and representatives have different penalty and refitting histories. The table compares implementation candidates, rather than isolating a causal effect of leaf type at identical support, initialization trajectory or hyperparameter budget.

Implementation and reproducibility

Pinned comparison. `scripts/compare_ten_models.py` selects and checks the ten entries; `docs/ten_model_comparison_v1/validation.json` records their metrics, source hashes, checkpoint hashes, layouts and measured timing. The frozen document input is `paper/sources/ten_model_comparison.json`. `scripts/check_comparison_working_points.py` checks both the scalar C++ and portable NumPy calibrations on all validation events; the report is `docs/ten_model_comparison_v1/working_point_checks.json`. The newest oblique/linear winner is case 50 of `configs/tao_hyper_scan_v1.json`, with its exact worker policy in `configs/hyper_scan_v1/case_50.json`. The axis cohort is pinned by `configs/axis_linear_v1.json`; its four regularized and refit policies are in `configs/axis_linear_v1/`. The axis refits run only the existing `leaf_only` stage, preserving axis cuts and the selected exact-zero leaf mask. The old 57.12% result is now a smaller representative, not the current oblique/linear performance winner. The historical v3 default remains Compact refit; no firmware resource winner has been selected.

5.3 Validation performance and parameter use

Figure 1 compares the selected ROCs; Fig. 2 shows their working points against scalar and multiplication budgets. The XGBoost reference reaches 55.48% signal efficiency at 1% background. The five-tree oblique/linear forest reaches 57.73% with 1275 nonzero scalars. At almost the same scalar count, five oblique/constant trees use 1250 and reach 55.40%. Their respective AUCs are 0.93090 and 0.92610, compared with 0.92056 for XGBoost. These are observed validation differences; model selection and training variation preclude an independent accuracy claim.

Linear leaf functions can compensate for a smaller or sparser partition. Compact refit reaches

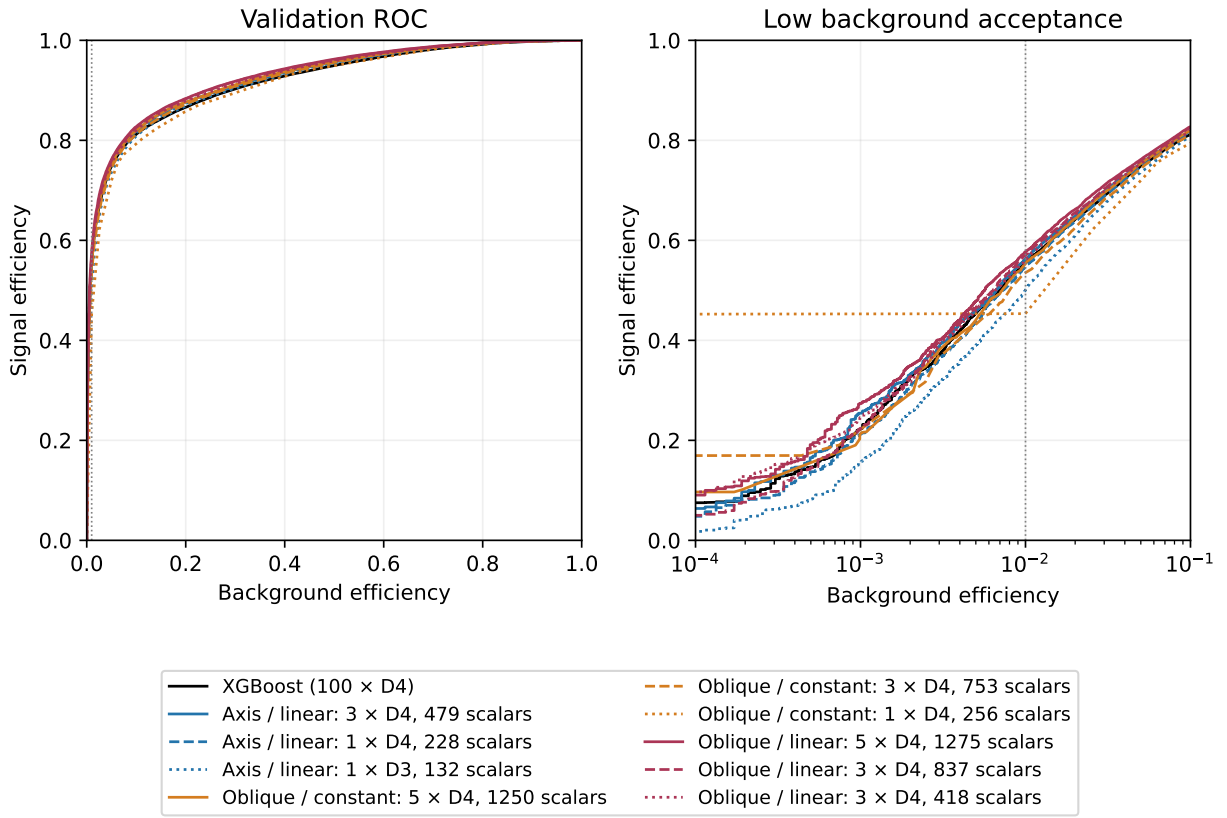


Figure 1: The ten selected configurations on the same validation partition. Colors identify families; line styles distinguish their representatives. The low-background panel resolves the primary working point. Numbers in the legend are learned nonzero scalar counts, with the axis-cut storage convention defined in the text.

56.20% with 418 nonzeros (379 after input-independent branch pruning), approximately three times fewer than the 1250-coefficient constant-leaf winner. This comparison includes both cut and leaf coefficients: a richer leaf increases its own arithmetic, but can reduce the overall number of retained cuts and weights. Conversely, axis-aligned routing can remain competitive when the leaves are expressive. A single axis-aligned depth-four tree reaches 54.74% using 228 learned nonzero scalars and 15 feature choices/direction bits. Three such trees reach 56.32% using 479 scalars and 45 discrete cut choices.

The distinction between interpolation and a realizable threshold matters most for coarse constant-leaf scores. The constant-leaf winner's interpolated 55.40% becomes 54.75% at a strict threshold respecting the 525-background event budget. The selected linear-leaf entries retain their displayed working-point efficiencies under the strict-threshold convention. A high AUC alone does not establish a better low-background selection. A strict threshold is tied to its numerical evaluator: changing even binary64 reduction order can move an event at the cut, so the working point must be checked again after a change of arithmetic, not only after quantization.

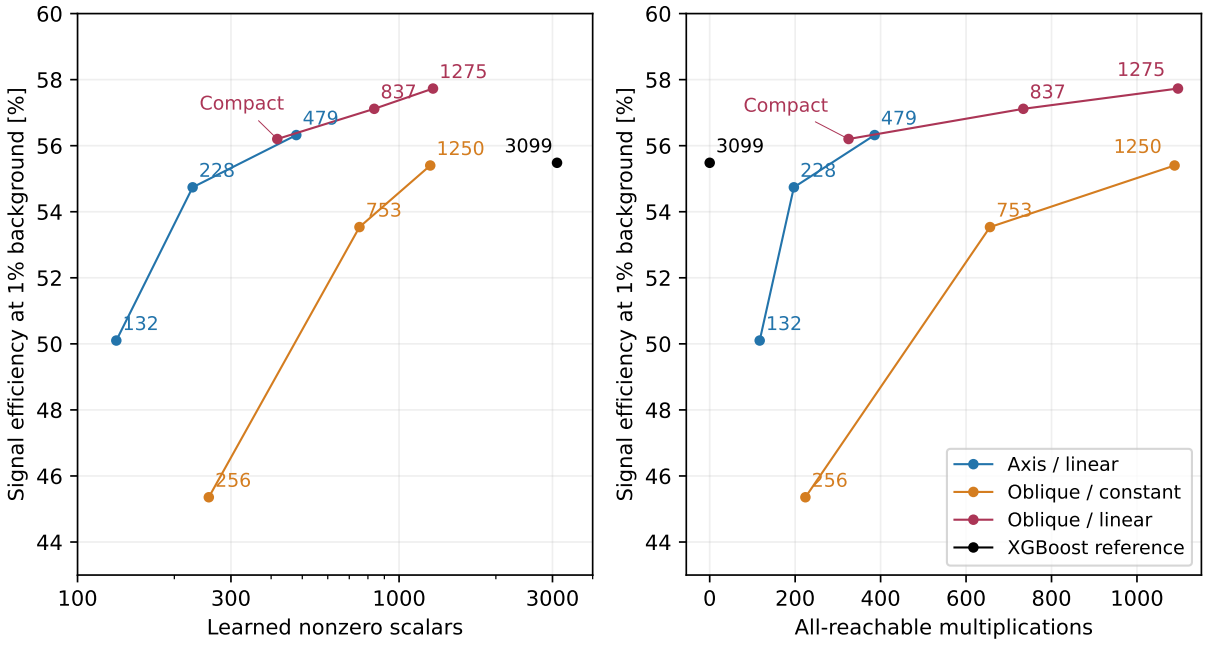


Figure 2: Representative validation operating points versus learned nonzero scalars (left) and all-reachable multiplications after exact-zero/unit-factor elimination (right). Lines only connect the selected representatives. Labels identify scalar counts; Compact denotes the 418-coefficient refit. Feature indices, direction bits, comparisons and additions are separate costs. XGBoost uses no general multiplications here, but still performs tree decisions and score aggregation.

5.4 Matched CPU evaluation and arithmetic workload

The C++ column in Table 8 uses one common sparse path-traversal reference for all ten models. Inputs and coefficients are binary64, with XGBoost inputs rounded to binary32 before traversal to preserve its native feature semantics. Exact-zero and unit-factor operations are eliminated; axis cuts use direct comparisons. All node and coefficient structures are loaded before timing. With one thread, one untimed warmup and nine timed repeats of the full 132,800-event batch, we report median wall time divided by event count. Parsing, preprocessing, allocation, input conversion and file I/O are excluded. The measurement host is a KVM guest reporting Intel Xeon Gold 6138 processors with a nominal 2.00 GHz clock; no fixed-frequency control is assumed. Compilation uses GCC 12.3.1, C++17, optimization level 3 and disabled fused multiply-add contraction. The results

measure bulk throughput, not single-event latency.

The axis three-tree depth-four model takes 0.101 microseconds per event, compared with 0.167 for Compact refit, 0.412 for the five-tree oblique/linear winner and 0.965 for the five-tree constant-leaf winner. The common reference takes 3.675 microseconds for XGBoost. Separately, the optimized native XGBoost library takes 1.478 microseconds per event on the same host and batch, including probability output. Thus the common kernel is a controlled reference, not the best achievable software implementation of every model. Differences from earlier NumPy and C++ prototype timings must not be read as changes in the algorithms' intrinsic cost.

The choice of hardware schedule changes the relevant arithmetic comparison. Axis three-tree depth-four routing requires no general multiplications, and its selected-path bound is 35 leaf products, versus 78 total products for Compact refit. Computing all reachable affine functions instead requires 386 products for the axis model and 325 for Compact refit. Their path comparison bounds are both 12. The axis model therefore has fewer sequential-path products, but more scalars (479 versus 418), more all-node products and additional discrete cut choices. Neither point uniformly dominates the other. XGBoost has zero general products because the learning rate is absorbed into its leaf values, yet has up to 400 path comparisons and 1499 internal decisions in an all-node evaluation. Multiplication count alone cannot rank all designs.

Implementation and reproducibility

One timing kernel and independent checks. `fpga/comparison/benchmark.cc` implements the shared sparse reference. `scripts/run_ten_model_comparison.sh` uses the pinned EL9/CMSSW environment; `scripts/compare_ten_models.py` prepares canonical models and validates predictions and every leaf assignment before recording results (16.6 million assignments across the ten validation predictions). The host was `clip-c2-48.cbe.vbc.ac.at`, with GCC 12.3.1, `-O3-std=c++17-ffp-contract=off`. All leaf assignments match exactly on validation. The largest XGBoost margin difference from native inference is 2.89×10^{-6}; each TAO margin differs by less than 10^{-9}. `operations` counts nonzero, non-unit feature factors after constant-route pruning. Older `sparsity_profile` counts retain unit products and are therefore slightly larger; the final table uses the new convention consistently. `paper/build_comparison_assets.py` regenerates the table from the frozen report; `--plot` additionally needs archived validation scores on CLIP.

6 Discussion and conclusions

The comparison supports three observations. First, expressive leaves can reduce the total coefficient and arithmetic budget needed for a useful working point. Compact refit's 418 scalars reach comparable validation performance to the 3099-scalar boosted reference, and exceed the selected 1250-scalar constant-leaf forest's working point. Second, inexpensive axis routing with linear leaves is competitive: the 479-scalar model reaches 56.32%, while a single 228-scalar tree reaches 54.74%. Third, the preferred candidate depends on implementation schedule. For the two candidates near 56.2%, the axis model uses fewer selected-path products; Compact refit uses fewer scalars and fewer all-node products. The larger oblique/linear model supplies the highest observed working point, 57.73%, at a higher resource budget. The final table records these distinct choices.

No single scalar count captures FPGA cost. Fixed unit cuts, nontrivial affine weights, comparisons, adders, sparse indices and leaf selection differ in implementation cost. Zero products do not make XGBoost free: its tree count still requires substantial routing and score aggregation. A path evaluator can reuse arithmetic but has sequential dependencies; an all-node pipeline can spend more resources to expose parallelism. Coefficient precision and constant-multiplier transformations can change either ranking. C++ throughput is useful software evidence, but does not determine FPGA clock, initiation interval or latency.

Training and selection also limit interpretation. The common objective and fixed data partitions support a consistent candidate comparison, but the families have different scan and refitting histories. Select–Freeze–Refit can recover shrinkage loss without restoring excluded directions; it cannot repair an inadequate selected support. Axis three-tree depth-three performance is worse at the primary working point than a smaller single depth-four tree, despite its slightly higher AUC. Model size is not a guarantee of low-background performance. Separation explains why one unpenalized axis refit was held, rather than being reported as a converged improvement. Existing conditional bootstrap intervals for the earlier subset are retained in Appendix E; no independent significance claim is made for the small differences between the new selected models.

This closes the floating-point candidate comparison and its supporting numerical implementation checks. Hardware evaluation requires a common target device, clock, interface and throughput requirement; validated feature, coefficient and accumulator formats; and a recalibrated realizable selection threshold after quantization. The scalar-leaf weighted-cut implementation is already numerically checked, whereas affine leaves require a corresponding hardware-backend extension. Synthesis must decide whether simpler routing or sparser all-node computation provides the better resource/latency tradeoff. Further test evaluation should retain disclosure of the historical XGBoost-only test check and use model choices fixed before inspecting that partition.

Implementation and reproducibility

Final handover. `fpga_handoff/CURRENT_COMPARISON.json` pins `fpga_handoff/comparison_v1/manifest.json` and its verification report. All ten table entries have exact standardized coefficients, golden vectors, metrics and training/configuration pointers. The three family winners and Compact refit have distinct roles; no synthesized hardware winner is selected. `docs/DORIAN_HANDOVER.md` is the entry point for Dorian. The historical v3 bundle and its default remain reproducible.

References

[1] Eibe Frank, Yong Wang, Stuart J. Inglis, Geoffrey Holmes, and Ian H. Witten. Using model trees for classification. *Machine Learning*, 32(1):63–76, 1998. doi: 10.1023/A:1007421302149. URL https://researchcommons.waikato.ac.nz/entities/publication/031c7d66-a7ac-4874-bc84-676e36d5c542.

[2] Miguel Á. Carreira-Perpiñán and Pooya Tavallali. Alternating optimization of decision trees, with application to learning sparse oblique trees. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/185c29dc24325934ee377cfda20e414c-Abstract.html.

[3] Suryabhan Singh Hada, Miguel Á. Carreira-Perpiñán, and Arman Zharmagambetov. Sparse oblique decision trees: A tool to understand and manipulate neural net features. arXiv:2104.02922, 2021. URL https://arxiv.org/abs/2104.02922. First posted in 2021; revised manuscript and journal version in 2023.

[4] Nicolai Meinshausen. Relaxed lasso. *Computational Statistics & Data Analysis*, 52(1):374–393, 2007. doi: 10.1016/j.csda.2006.12.019. URL https://people.math.ethz.ch/~nicolai/relaxo.pdf.

[5] OpenML. hls4ml_lhc_jets_hlf, dataset 42468. Dataset catalogue. URL https://www.openml.org/d/42468. The project uses its archived feature arrays and fixed splits, rather than a new split from the catalogue.

[6] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*,

pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL https://arxiv.org/abs/1603.02754.

[7] HEPHY Analysis Software. Oblique tree benchmark, 2026. URL https://github.com/HephyAnalysisSW/oblique-tree-benchmark. Implementation, frozen v3 FPGA handover and dated research records; exact source fingerprints accompany this note.

[8] Sioni Summers, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Duc Hoang, Sergo Jindariani, Edward Kreinar, Vladimir Loncar, Jennifer Ngadiuba, Maurizio Pierini, Dylan Rankin, Nhan Tran, and Zhenbin Wu. Fast inference of boosted decision trees in FPGAs for particle physics. *Journal of Instrumentation*, 15:P05026, 2020. doi: 10.1088/1748-0221/15/05/P05026. URL https://arxiv.org/abs/2002.02534.

A Support-selection and refit results

For Compact refit, a converged three-tree parent was continued along a split-penalty path $\lambda_s = 3{\cdot}10^{-5}, 10^{-4}, 3{\cdot}10^{-4}, 10^{-3}, 3{\cdot}10^{-3}$, with $\lambda_\ell = 10^{-5}$. The terminal fit at the final penalty supplied 418 coefficients. Leaf-only refitting raised validation efficiency from 54.59% to 54.81%. Full alternating refitting converged after 15 epochs at 55.98%; its best saved validation snapshot was epoch 8 at 56.20%. The selected model is this earlier snapshot, with the same 418 coefficients. An additional one-tree oblique reference, retained in the historical uncertainty subset but not among the final ten representatives, was trained without L1 and without SFR.

Table 1 and Fig. 3 summarize ten parent supports. Six full refits completed, two were held for separation and two for an inconclusive linear-program timeout. The higher-efficiency model is a completed leaf-only refit at 57.12%, improving its own parent by about 0.21 percentage points. Its full stage was held and is excluded from selection. Stronger penalization does not consistently improve the accuracy–size tradeoff; refitting cannot restore a direction removed by support selection.

Table 1: Fixed-support refitting study. Efficiencies are percentages at 1% background. All models have three depth-four trees except the starred one-tree case. Parent and leaf columns are terminal stage results; best is the best eligible saved leaf/full refit. Held full stages contribute no candidates. The support budget is the parent's exact nonzero count.

λ_s/λ_ℓ	Support	Parent	Leaf	Best	Full stage
0.0001/0.0001	520	55.58	55.82	56.49	complete
0.0003/1e-5	675	56.98	56.74	56.74	timeout
1e-5/3e-5	837	56.91	57.12	57.12	timeout
0.0003/0.0003	394	54.48	54.85	54.85	separated
1e-5/1e-5*	403	55.20	55.06	55.08	complete
0.001/1e-5	561	56.06	56.32	56.46	complete
3e-5/3e-5	777	56.62	56.75	56.75	complete
1e-5/1e-5	889	56.40	56.45	56.74	complete
0.003/1e-5	418	54.59	54.81	56.20	complete
0.001/0.001	317	51.27	52.52	52.52	separated

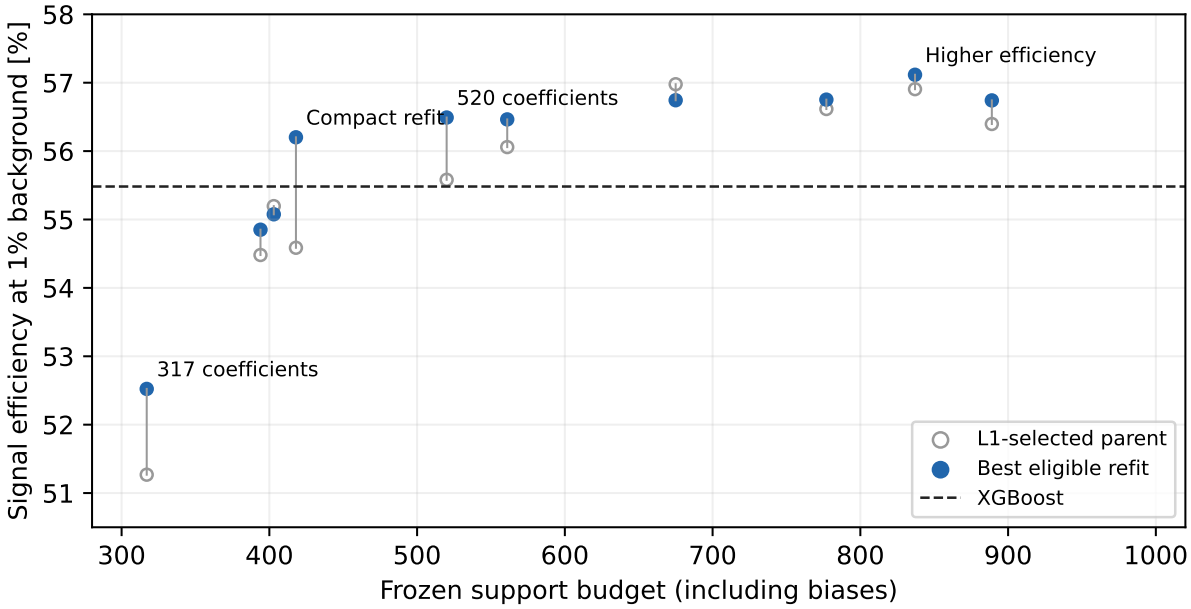


Figure 3: L1-selected terminal parents and their best eligible fixed-support refits. Each vertical segment keeps the support budget fixed. Only refits from eligible stages contribute; the penalty settings are given in Table 1.

Implementation and reproducibility

Historical refit inputs. This supporting study uses the v3 model bundle. `fpga_handoff/CURRENT_MODEL.json` selects `fpga_handoff/v3/compact_refit_3t_d4/`. The 837-coefficient alternative is `highest_efficiency_3t_d4`; the one-tree reference is `single_tree_1t_d4`. All exact source snapshots and masks are recorded in the handover manifest. A later 57.24% full-refit diagnostic for the higher-efficiency support is held and not an eligible model.

Compact training chain. Start from the v2 `winner_3t_d4` coefficients, run `configs/l1_path_v1/control.json`, then the split path `split_01.json` through `split_05.json` in `configs/l1_path_v1/`. The manifest is `configs/tao_l1_path_v1.json`. The final terminal parent feeds `configs/support_refit_scan_v1/split300.json`. Parent training uses `scripts/train_tao_fully.py`; SFR uses `scripts/train_tao_support_refit.py`, `scripts/run_support_refit_scan_case.py` and `scripts/support_refit_artifacts.py`. Exact refit reproduction begins with the archived hash-pinned terminal parent, rather than a fresh fit at the final penalties. The selected checkpoint hash is

`e89b8923fba66f5df02046e04a1828a8ea4c56f60c1d7b201a2a2f92a0ab00fa`

Budgets. The penalty path starts with local/joint/split iteration budgets 300/250/300 and logs repeated chunks or adaptive increases. Compact refit uses 400/400/800, L-BFGS-B tolerances `ftol=1e-12`, `gtol=1e-8`, at most 100 joint-fit chunks and 200 epochs per stage. A 2,700-second allocation checkpoints with return code 75. A mild-control router required a documented 800-to-4,000 iteration-budget amendment and converged after 2,028 auxiliary iterations; this changed no objective or stopping tolerances. See `scripts/amend_support_solver_budget.py` and `docs/support_refit_scan_budget_amendments.json`.

Snapshot layers. The refit study was frozen on 25 October 2026. The final comparison is separately frozen on 29 October and includes the newly converged five-tree architecture winner at 57.73%. Historical dated records are retained but do not supersede this comparison. `src/tao/uncertainty.py` supplies the earlier bootstrap; `scripts/build_paper_inputs.py` regenerates that frozen supporting subset.

B Axis-aligned linear-leaf cohort

All four CART-initialized source configurations converged under the common objective/KKT/routing stability rules, with $\lambda_\ell = 10^{-5}$ and $\lambda_s = 0$. Their frozen-support, fixed-cut unpenalized leaf refits produced three converged stages and one separation outcome. Table 2 reports the best eligible checkpoint for each layout. The three-tree depth-four entry therefore uses its regularized source, and the three-tree depth-three entry also retains its regularized checkpoint because its eligible refit did not improve the primary validation metric.

Table 2: Complete axis/linear cohort. Scalar slots exclude fixed unit routing weights; nonzeros include learned leaf values and active thresholds. Index counts require an additional direction bit per nonconstant cut. The refit column reports the unpenalized stage's outcome, not the source's convergence.

Layout	Scalar slots	Nonzeros	Indices	ϵ_s [%]	AUC	Refit
1×D3	143	132	7	50.10	0.92020	complete
1×D4	287	228	15	54.74	0.92387	complete
3×D3	429	274	21	53.83	0.92414	complete
3×D4	861	479	45	56.32	0.92576	separated

A separated unpenalized fit is held because coefficients can grow along a recession direction without a finite minimizing parameter vector, as described in Section 4. Its regularized source can remain a valid candidate; the held stage supplies no selected result.

Implementation and reproducibility

Exact source/refit states, selected epochs, hashes and parameter counts are in `docs/axis_linear_v1/status.json`; the frozen paper copy is `paper/sources/axis_linear_study.json`. The four source policies are `configs/axis_linear_v1/case_0.json` through `case_3.json`; their hash-pinned terminal parents feed `refit_0.json` through `refit_3.json`. `scripts/monitor_axis_study.py` resumed only clean wall-time exits and stopped when every case had a converged or explicitly held refit outcome. No source in this four-case study remains incomplete.

C Constant-leaf cohort

Constant predictions simplify the hardware leaf stage and permit comparison with scalar-leaf FPGA tree architectures [8]. To study the accuracy cost, we scan $T \in \{1, 3, 5\}$ trees, $D \in \{2, 3, 4\}$ and $\lambda_s \in \{0, 10^{-7}, 10^{-5}\}$, giving 27 configurations. Each uses the same CART initialization seed, training partition and numerical stopping rules as the factorial architecture study. Constant leaves have no feature weights, so their nominal leaf penalty does not affect Eq. (10). We select the saved checkpoint with the highest validation efficiency at 1% background from each converged source run, and then select among penalties for each layout.

All 27 source runs have converged. Table 3 reports the best selected penalty per layout. The interpolated efficiency follows the common ROC convention. We also report a strict threshold on the binary64 signal margin that accepts at most 525 background events. This distinction matters for shallow constant trees: a single depth-D tree has at most 2^D score levels, and even its highest level can exceed the background budget. Connecting ROC vertices then gives an interpolated efficiency that a deterministic threshold cannot realize.

The three selected depth-four layouts supply the constant-leaf rows in the final comparison. Detailed source/checkpoint provenance remains in the archived cohort.

Table 3: Constant-leaf forests: the best eligible penalty per layout. ϵ_s is interpolated at 1% background; $\epsilon_s^{\text{strict}}$ uses a realizable strict margin threshold with background efficiency at most 1%. Both coefficient counts include biases.

Layout	λ_s	P_{dense}	P_{nz}	ϵ_s [%]	$\epsilon_s^{\text{strict}}$ [%]	AUC
$1 \times$ D2	10^{-7}	55	55	14.98	0.00	0.88179
$1 \times$ D3	10^{-5}	127	126	38.61	0.00	0.90357
$1 \times$ D4	0	271	256	45.36	45.33	0.91247
$3 \times$ D2	10^{-5}	165	129	30.72	0.00	0.90087
$3 \times$ D3	0	381	336	49.01	38.81	0.91763
$3 \times$ D4	0	813	753	53.54	53.30	0.92318
$5 \times$ D2	10^{-7}	275	275	47.77	47.01	0.91283
$5 \times$ D3	10^{-7}	635	615	50.94	50.31	0.92176
$5 \times$ D4	0	1355	1250	55.40	54.75	0.92610

D Fixed-point feasibility checks

D.1 A fixed-point XGBoost reference

As a first numerical implementation check, we emulate the XGBoost reference with signed fixed-point inputs, thresholds and scores. A format with total width W_b and integer width I_b has $F_b = W_b - I_b$ fractional bits and representable range $[-2^{I_b-1}, 2^{I_b-1} - 2^{-F_b}]$. Define its quantizer by

$$Q_{W_b,I_b}(u) = \text{clip}_{W_b,I_b}\left(2^{-F_b}\,\text{round}_{\text{even}}(2^{F_b}u)\right), \tag{41}$$

where rounding chooses the nearest integer with ties to even, and clipping saturates at the format's endpoints. We quantize inputs and thresholds before evaluating the strict-left XGBoost cuts. Let $s_{t\ell}$ denote a scalar leaf contribution with the learning rate absorbed, and s_0 the initial margin. These values use the score format; tree scores are combined by a balanced adder tree with casts to that format after additions. Its result is added to the quantized initial margin and cast once more. No sigmoid is needed for selection.

The maximum absolute training input is 8.96 and the largest absolute tree threshold is 4.73, so we use $I_b = 5$ for inputs and thresholds. For the $T = 100$ tree scores, define

$$A = |s_0| + \sum_{t=1}^{T} \max_{\ell} |s_{t\ell}| \simeq 15.09. \tag{42}$$

At the coarsest fractional width $F_{\min} = 4$, rounding the T leaf values and initial margin can add at most $(T+1)2^{-F_{\min}-1}$ to this bound. Their quantized sum and all partial sums therefore fit within an absolute bound of 18.25, covered by $I_b = 6$ for scores. With no overflow, sums of values on this common fixed-point grid introduce no further fractional rounding. No validation input lies outside the chosen input range. We vary the common fractional width while keeping these integer widths fixed. Table 4 shows the resulting validation performance. The 17/18-bit input/score pair retains the reference working-point efficiency; the coarsest 9/10-bit pair loses performance and produces more substantial score ties. Small positive changes in the interpolated point are not interpreted as an established accuracy gain.

Independent C++ executions agree exactly for the tested fixed-point outputs. This establishes numerical agreement of the emulators; FPGA resource use, latency and initiation interval require synthesis for a specified device, clock and interface.

Table 4: XGBoost fixed-point emulation on the same validation partition. Widths include the sign bit; input and threshold formats are identical. The native reference has $\epsilon_s = 55.48\%$ and AUC 0.92056. The strict-threshold column uses background efficiency at most 1%.

Input bits	Score bits	ϵ_s [%]	$\epsilon_s^{\text{strict}}$ [%]	AUC
23	24	55.48	55.48	0.92056
17	18	55.49	55.49	0.92056
15	16	55.46	55.45	0.92056
13	14	55.56	55.53	0.92048
11	12	55.47	55.29	0.92040
9	10	52.05	50.85	0.91760

Implementation and reproducibility

Tested chain. `docs/conifer_xgboost_v1/README.md` specifies the EL9 container, CMSSW workspace, dependency pins and commands. Conversion and HLS C++ generation use conifer 1.11; the axis-aligned reference is also evaluated with the actual CMSSW conifer 1.7 external. Its bridge rejects oblique weights. The full validation compares 13,280,000 native/exported leaf IDs, plus boundary checks. Each of eight C++ profiles is checked on 512 HLS vectors, including all 324 frozen XGBoost handover vectors. Every fixed-point comparison is exact. `fpga/conifer/compat.py` reconciles native binary32 thresholds, leaf values, initial margin and actual depth; upstream conversion is retained as a diagnostic. It is important to preserve XGBoost's input rounding and strict-left equality convention. The score-type settings in the table are `AP_RND_CONV,AP_SAT`; axis weights are exact one-hot constants.
`docs/conifer_xgboost_v1/validation.json` contains full provenance and per-profile metrics. `scripts/validate_xgboost_conifer.py` reads training inputs for range selection and validation arrays for evaluation; test arrays are not loaded. Generated projects use conifer's default device/clock only as export placeholders. No FPGA synthesis has been run, and CPU timings that include file I/O are not reported as hardware latency.

D.2 Oblique-leaf feasibility check

The following supporting precision check concerns the previously frozen five-tree constant-leaf winner and three-tree linear Compact refit only. It does not quantize all ten final representatives or the new axis/linear cohort. For an exploratory reduced-precision comparison, each cut is first multiplied by a positive power of two, $w'_{tv} = 2^{k_{tv}} w_{tv}$ and $b'_{tv} = 2^{k_{tv}} b_{tv}$. This preserves its real-arithmetic decision; the exported binary64 scaling is constrained to retain every nonzero coefficient, and validation leaf assignments are checked before quantization. We use the nearest-even, saturating quantizer of Eq. (41), with 10 integer bits for inputs, thresholds and cut accumulators, 6 for weights, and 16 for scores and leaf biases, all including sign. Every type uses the same fractional width F. Cast inputs and coefficients precede multiplication; cut products accumulate in feature order, with a cast to the cut-accumulator format after each addition. For a linear leaf, products accumulate similarly in the score format, followed by the leaf bias. The negative tree outputs are summed in the score format. Integer widths are conservatively checked against absolute partial-sum bounds in the observed training box; they have not been minimized separately for the two candidates.

At $F = 12$, both architectures remain within 0.02 percentage points of their floating-point interpolated working-point efficiency (Table 5). For constant leaves, independent fixed-point path traversal and the existing weighted-cut backend agree exactly on all validation scores. The generated all-node HLS C++ implementation also agrees exactly on a 512-vector check at $F = 12$. The linear-leaf evaluation uses an independent affine-leaf reference; a corresponding HLS backend extension and synthesis remain to be completed. These checks establish numerical feasibility at common precision,

Table 5: Common-format fixed-point emulation of the two selected leaf architectures. Efficiencies are percentages at 1% background. Total widths are $F + 10$ input/threshold bits, $F + 6$ weight bits and $F + 16$ score bits.

Leaf type	F	ϵ_s [%]	$\epsilon_s^{\text{strict}}$ [%]
Constant	18	55.40	54.75
Constant	12	55.40	54.75
Constant	8	55.18	54.38
Linear	18	56.20	56.20
Linear	12	56.21	56.21
Linear	8	56.00	56.00

rather than an optimized FPGA resource comparison.

Implementation and reproducibility

`docs/leaf_competition_v1/README.md` records this earlier two-candidate experiment. `scripts/run_leaf_competition.sh` and `fpga/leaf_competition/emulate.cc` reproduce its independent sparse reference. Constant leaves use unmodified conifer 1.11 weighted-cut/scalar-leaf backends; the old CMSSW 1.7 external is axis-aligned only. Affine leaves still require a conifer HLS backend extension. This feasibility check is separate from the final common timing kernel and does not establish synthesized resources.

E Conditional uncertainty of an earlier subset

These previously frozen intervals concern Compact refit, the 837-coefficient leaf refit and a one-tree oblique reference. They are not intervals for all ten current representatives; in particular they do not assess the new axis/linear models or five-tree performance winner. In this supporting table, "Higher efficiency denotes the 837-coefficient refit.

A paired class-stratified bootstrap with 1,000 replicates, seed 20261025, recomputes the interpolated working point in each replicate. Table 6 gives percentile intervals for the difference from XGBoost. The Compact interval includes zero. These intervals condition on the selected models and account for neither hyperparameter/checkpoint selection nor training-seed variation; they are not independent test claims.

Table 6: Conditional validation efficiency difference from XGBoost, in percentage points (pp), with paired 95% percentile intervals.

Model	Difference [pp]	Interval [pp]
Compact refit	+0.72	[-0.43, +1.52]
Higher efficiency	+1.63	[+0.65, +2.55]
One-tree reference	+0.15	[-1.08, +0.97]

F Input feature order

Table 7 defines the coordinates r_f of the archived feature vector, using the source dataset's feature names. All displayed cuts use their standardized counterparts x_f from Eq. (39).

Table 7: Feature-coordinate convention. Names identify columns in the high-level jet dataset [5].

f	Dataset feature	f	Dataset feature
1	`zlogz`	9	`d2_a1_b1_mmdt`
2	`c1_b0_mmdt`	10	`d2_a1_b2_mmdt`
3	`c1_b1_mmdt`	11	`m2_b1_mmdt`
4	`c1_b2_mmdt`	12	`m2_b2_mmdt`
5	`c2_b1_mmdt`	13	`n2_b1_mmdt`
6	`c2_b2_mmdt`	14	`n2_b2_mmdt`
7	`d2_b1_mmdt`	15	`mass_mmdt`
8	`d2_b2_mmdt`	16	`multiplicity`

G Actual learned functions

Actual cuts illustrate the mixture of axis-aligned, oblique and constant nodes. In Compact refit, using the standardized feature indices of Table 7, the first tree contains

$$h_{1,\varnothing}(x) \simeq x_4 + 0.4216204 \tag{43}$$

$$h_{1,L}(x) = 0 \tag{44}$$

$$\begin{aligned} h_{1,R}(x) \simeq\ & 0.088106595x_4 + 0.0025365055x_6 + 0.17735909x_{11} \\ & + 0.031223362x_{15} + 0.10064081x_{16} + 0.019058664 \end{aligned} \tag{45}$$

Coefficients marked $\simeq$ are rounded to eight significant digits. At the root an event goes right if $x_4 > -0.4216204$. Otherwise it reaches L, whose constant cut sends it left again. Hence the entire LR subtree is unreachable. The root's right child combines five input features.

For the second tree, another learned cut is

$$h_{2,L}(x) \simeq -2.3776136 \cdot 10^{-11} x_{16} - 3.1703281 \cdot 10^{-11} \tag{46}$$

Its small scale does not make it uninformative: it routes right approximately when $x_{16} < -1.3334$. Setting both coefficients to zero would turn it into a constant left route. This example motivates distinguishing exact sparsity from coefficient magnitude before quantization.

Implementation and reproducibility

Print the actual functions. `TreePrinter` in `src/tao/printing.py` provides `print_tree`, `format_tree`, `format_forest` and `records`. The CLI `scripts/print_tao_tree.py` prints every cut and affine leaf, including unreachable branches. It is read-only and separate from pinned fitting code. For example, after setting the environment described below:

```
import json
from pathlib import Path
from tao.printing import TreePrinter
spec = json.loads(Path(
    "fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json"
).read_text())
TreePrinter(spec, precision=17).print_tree(0)
```

The complete 17-digit record is `generated/compact_forest_full_precision.txt`; the annotated

supplement prints all three trees at nine display digits. Only exact-zero terms are omitted. Independent tests parse the printed expressions, recover all 418 nonzero coefficients, count 379 reachable coefficients, and compare predictions with the portable evaluator. They also check equality routing, constant leaves, tiny terms and non-mutation. The printer tests also preserve the source model and its exact-zero support.

Implementation and reproducibility

Environment and entry points. Training and input regeneration use Python 3.10 with the pinned `requirements.txt`: NumPy 1.26.4, SciPy 1.15.2, scikit-learn 1.7.2, XGBoost 3.2.0, PyYAML 6.0.3 and matplotlib 3.10.7. The reference CLIP checkout is `/users/robert.schoefbeck/CMS/tree_benchmark`; its Conda environment is `tree-benchmark`. On CLIP use the conda-forge CPU XGBoost build, since the PyPI Linux wheel requires newer glibc than the reference login environment. No GPU is needed.

```
source /software/f2022/software/mamba/24.3.0-0/etc/profile.d/conda.sh
conda activate tree-benchmark
cd /users/robert.schoefbeck/CMS/tree_benchmark
export PYTHONPATH="$PWD/src:$PWD" PYTHONDONTWRITEBYTECODE=1
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
python scripts/print_tao_tree.py \
  --model fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json \
  --precision 17 --output /tmp/compact_forest.txt
```

For a new training account or checkout, the complete environment-creation and controlled refit recipe is in `docs/DORIAN_HANDOVER.md`. The datasets and pickle archives are deliberately not in Git. Exact reproduction of the refit starts from the hash-pinned terminal parent; rebuilding its entire L1 path can produce differently serialized checkpoint bytes and requires a new explicit source hash/run name. Existing named runs must not be resumed under changed training code or stopping policies. The standalone `paper/` folder, however, contains all generated text/tables and figures needed to compile this PDF without datasets, model pickles or a Python training environment.

Implementation map.

Object / operation	Entry point
Axis cut search	`AxisTAOClassifier._split_update` in `src/tao/axis.py`
Matched C++ comparison	`fpga/comparison/benchmark.cc`; `scripts/compare_ten_models.py`
Final table generation	`paper/build_comparison_assets.py`
Routing, logits, objective	`HardTree`; `HardTAOClassifier` in `src/tao/hard.py`
Weighted leaf optimization	`fit_leaf_parameters`; `ce_value_gradient`
Joint leaf design/refit	`_joint_design`; `global_refit`
Reduced split proposals	`_hard_split_update`; `FrozenSupportTAO._split_update`
Sorted hard threshold search	`best_threshold` in `src/tao/split_search.py`
Node-resumable epoch	`begin_epoch`; `step_epoch` in `src/tao/epoch.py`
Masks and separation	`freeze_support`; `continue_full_refit`; `separation_check`
Stopping and exact-zero work counts	`leaf_kkt_residual`; `convergence_check`; `sparsity_profile`
Read-only equations	`TreePrinter.print_tree` in `src/tao/printing.py`
Portable inference / firmware export	`export_model`; `predict_margin`; `cpp_header` in `src/tao/portable.py`

Frozen coefficients and verification. `fpga_handoff/CURRENT_COMPARISON.json` pins all ten final entries in `fpga_handoff/comparison_v1/`. Each TAO candidate has standardized `model.json`, hexadecimal `model_coefficients.h`, ordered golden inputs and expected margins/leaf indices. The bundle is standardized-only; use its input contract and training scaler. Historical

v3 additionally retains raw-folded variants for its original candidates. Do not mix those input spaces.
scripts/verify_comparison_handoff.py needs Python's standard library and a C++17 compiler. It checks every file hash, compiles the nine TAO references, and independently walks exported XGBoost on its golden vectors. The full common C++ benchmark separately checks every validation leaf assignment against the canonical TAO or native XGBoost inference. Reproduction in a new checkout:

```
python3 scripts/verify_comparison_handoff.py --compiler c++
# On CLIP, use EL9 because the native GCC is too old:
scripts/conifer_container.sh bash -c \\
 'source scripts/conifer_runtime.sh && workspaces/python/conifer/bin/python scripts/
    ↪ verify_comparison_handoff.py --compiler g++'
# Reproduce the measured comparison with archived data/checkpoints:
scripts/run_ten_model_comparison.sh
# Rebuild its document assets:
python3 paper/build_comparison_assets.py --plot
make -C paper
```

The plot command requires archived score files on CLIP; rebuilding both PDFs from the complete paper/ folder requires only TeX and the included assets. scripts/freeze_final_comparison.py created the immutable bundle once and refuses to overwrite it. The old model bundles, pointer and verification reports remain available for historical reproduction. Fixed-point checks cover the stated earlier subsets; no synthesis or full ten-model quantization claim is made. Full training and configuration pointers are in docs/DORIAN_HANDOVER.md and the new manifest.

Implementation supplement: complete forest printout

Implementation and reproducibility

This record is generated by `TreePrinter.format_tree` from the exact v3 standardized model. It lists every internal decision and every leaf output in preorder, with heap indices and root-to-node paths. A node labelled unreachable is excluded only by an input-independent constant ancestor, not by absent training or validation examples. Numeric values are displayed with nine significant digits. The full 17-digit record is checked in as `generated/compact_forest_full_precision.txt`. These listings are for inspection; implementation should consume the frozen JSON or hexadecimal header. In all trees, a leaf output contributes to the background logit; the signal margin is the *negative sum* of the three selected outputs.

Reading the record. A path such as `RLLR` means right, left, left, right from the root. This path is the scientific node or leaf label. The extra `N` and `L` labels are internal-node and leaf-array indices; `heap` is a storage index. They have no mathematical role. Trees are numbered from zero in the file and from one in the scientific text.

Implementation and reproducibility

Tree 0 (scientific tree $t = 1$).

```
TREE 0: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_0_0(x) = 1 * c1_b2_mmdt + 0.421620399
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_0_1(x) = 0
    <= 0 -> N3; > 0 -> N4; CONSTANT: always LEFT

    N3 [path=LL; heap=3; reachable]
      h_0_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT

      N7 [path=LLL; heap=7; reachable]
        h_0_7(x) = 0.318308228 * c1_b0_mmdt + 0.330704464
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_0_0(x) = 0.723115375 * zlogz - 0.424343964 * c1_b0_mmdt - 0.446947898 * c1_b1_mmdt -
↪ 0.415001941 * c2_b1_mmdt - 0.139216682 * d2_b1_mmdt - 0.139216682 * d2_a1_b1_mmdt + 0.54944101 *
↪ m2_b1_mmdt - 0.495073946 * m2_b2_mmdt + 0.059415836 * n2_b1_mmdt + 0.858013396 * multiplicity -
↪ 1.35727329

        L1 [path=LLLR; heap=16; reachable]
          f_0_1(x) = 1.51929697 * zlogz + 0.669978191 * c1_b0_mmdt - 3.56315299 * c1_b2_mmdt +
↪ 0.613457803 * c2_b1_mmdt - 0.396278128 * d2_b1_mmdt + 0.152304141 * d2_b2_mmdt - 0.396278128 *
↪ d2_a1_b1_mmdt - 0.199974866 * m2_b2_mmdt + 0.980332415 * n2_b1_mmdt - 0.420497004 * n2_b2_mmdt +
↪ 0.396719294 * mass_mmdt + 1.34144785 * multiplicity - 1.72085229

      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_0_8(x) = 0
        <= 0 -> L2; > 0 -> L3; CONSTANT: always LEFT

        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_0_2(x) = 0

        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_0_3(x) = 0

    N4 [path=LR; heap=4; UNREACHABLE (constant ancestor)]
      h_0_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT

      N9 [path=LRL; heap=9; UNREACHABLE (constant ancestor)]
        h_0_9(x) = -0.0491155824 * c1_b0_mmdt + 0.000379329885
        <= 0 -> L4; > 0 -> L5

        L4 [path=LRLL; heap=19; UNREACHABLE (constant ancestor)]
          f_0_4(x) = 0

        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_0_5(x) = 0
```

```
      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_0_10(x) = 1.75931373e-09 * c1_b0_mmdt - 3.92622475e-09 * m2_b2_mmdt + 1.22858604e-07 *
↪ multiplicity - 1.097723e-07
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_0_6(x) = 0
        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_0_7(x) = 0
  N2 [path=R; heap=2; reachable]
    h_0_2(x) = 0.0881065951 * c1_b2_mmdt + 0.0025365055 * c2_b2_mmdt + 0.17735909 * m2_b1_mmdt +
↪ 0.0312233619 * mass_mmdt + 0.100640807 * multiplicity + 0.0190586642
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_0_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_0_11(x) = -9.45167785e-16 * d2_b1_mmdt - 5.84426162e-16
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_0_8(x) = 0.472450802 * zlogz + 0.0989313467 * c1_b0_mmdt - 0.831430736 * c1_b2_mmdt -
↪ 0.431555721 * c2_b2_mmdt + 0.0150358961 * d2_b1_mmdt + 0.648844514 * d2_b2_mmdt + 0.0150358961 *
↪ d2_a1_b1_mmdt - 3.09641516 * d2_a1_b2_mmdt + 1.49849996 * m2_b1_mmdt + 1.81153643 * n2_b1_mmdt -
↪ 0.893777123 * n2_b2_mmdt + 0.182450792 * multiplicity - 2.83938374
        L9 [path=RLLR; heap=24; reachable]
          f_0_9(x) = 0.232327574 * zlogz - 0.23205911 * c1_b0_mmdt - 0.516471161 * c1_b1_mmdt -
↪ 0.848527306 * c2_b1_mmdt - 1.97477014 * c2_b2_mmdt + 1.42114491 * d2_b2_mmdt + 0.536602443 *
↪ m2_b1_mmdt + 0.651756707 * mass_mmdt - 1.64666395
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_0_12(x) = 9.89173599e-11 * c1_b2_mmdt + 1.79437153e-09 * c2_b1_mmdt + 1.81919634e-09 *
↪ c2_b2_mmdt + 1.04854961e-09
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_0_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_0_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_0_6(x) = 0.164748862 * c1_b2_mmdt + 0.185195199 * c2_b2_mmdt + 0.271888733 * d2_a1_b2_mmdt +
↪ 1.06018366 * mass_mmdt - 0.265110937 * multiplicity - 1.32289565
      <= 0 -> N13; > 0 -> N14
      N13 [path=RRL; heap=13; reachable]
        h_0_13(x) = 0.635416434 * c1_b2_mmdt - 0.475835509 * multiplicity + 0.00561843162
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; reachable]
          f_0_12(x) = 0.000756449729 * zlogz + 0.576353531 * c1_b0_mmdt + 1.62064775 * c1_b1_mmdt -
↪ 2.48681119 * c1_b2_mmdt - 2.10518311 * c2_b1_mmdt + 1.37970957 * c2_b2_mmdt - 0.0135562553 *
↪ d2_b1_mmdt + 0.338362995 * d2_b2_mmdt - 0.0135562553 * d2_a1_b1_mmdt - 1.72149735 * d2_a1_b2_mmdt -
↪ 0.38025814 * m2_b1_mmdt - 0.351321056 * m2_b2_mmdt + 2.69042188 * n2_b1_mmdt - 0.919390109 *
↪ n2_b2_mmdt + 0.32251432 * mass_mmdt + 0.582538651 * multiplicity - 3.45473882
        L13 [path=RRLR; heap=28; reachable]
          f_0_13(x) = -0.140117186 * c1_b2_mmdt + 0.652505422 * c2_b1_mmdt + 0.253434919 * d2_b1_mmdt -
↪ 0.602112106 * d2_b2_mmdt + 0.253434919 * d2_a1_b1_mmdt - 0.527210682 * d2_a1_b2_mmdt - 0.00193453091 *
↪  n2_b2_mmdt - 1.64156297 * mass_mmdt + 0.0192123329 * multiplicity - 2.68453555
      N14 [path=RRR; heap=14; reachable]
        h_0_14(x) = 0
        <= 0 -> L14; > 0 -> L15; CONSTANT: always LEFT
        L14 [path=RRRL; heap=29; reachable]
          f_0_14(x) = -0.727395772 * zlogz - 1.0021119 * c1_b0_mmdt - 1.77913852 * c1_b1_mmdt +
↪ 1.29863387 * c1_b2_mmdt - 5.22419531 * c2_b1_mmdt + 1.67449112 * c2_b2_mmdt + 0.609159654 *
↪ d2_b1_mmdt - 1.14240277 * d2_b2_mmdt + 0.609159654 * d2_a1_b1_mmdt + 0.462711749 * d2_a1_b2_mmdt -
↪ 0.379623445 * m2_b1_mmdt - 1.49230987 * m2_b2_mmdt + 1.44153754 * n2_b1_mmdt - 4.67940637 * mass_mmdt
↪  + 1.29820647 * multiplicity + 5.68806483
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_0_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Implementation and reproducibility

Tree 1 (scientific tree $t = 2$).

```
TREE 1: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_1_0(x) = 1 * c1_b2_mmdt + 0.421515048
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_1_1(x) = -2.3776136e-11 * multiplicity - 3.17032814e-11
    <= 0 -> N3; > 0 -> N4

    N3 [path=LL; heap=3; reachable]
      h_1_3(x) = -0.0032891119 * zlogz + 0.0142175742 * d2_b1_mmdt + 0.00121827303 * d2_a1_b1_mmdt -
↪ 0.000244599886 * m2_b2_mmdt - 0.00553888952 * n2_b2_mmdt + 0.295132576 * mass_mmdt + 0.0218087714
      <= 0 -> N7; > 0 -> N8

      N7 [path=LLL; heap=7; reachable]
        h_1_7(x) = 0.0213173363 * zlogz + 0.612901764 * c1_b0_mmdt - 0.0606512036
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_1_0(x) = 1.02493155 * zlogz + 0.475780074 * c1_b0_mmdt - 3.46865641 * c1_b1_mmdt +
↪ 1.39660226 * c1_b2_mmdt + 0.0808524935 * d2_b1_mmdt + 0.0625866107 * d2_b2_mmdt + 0.0808524935 *
↪ d2_a1_b1_mmdt + 0.265262813 * d2_a1_b2_mmdt + 0.0469595498 * m2_b1_mmdt - 0.00734786374 * m2_b2_mmdt +
↪  3.49945192 * mass_mmdt - 0.346146245

        L1 [path=LLLR; heap=16; reachable]
          f_1_1(x) = 2.42880675 * zlogz + 3.52193453 * c1_b0_mmdt - 2.27198818 * c1_b1_mmdt - 0.640806627
↪  * c1_b2_mmdt + 0.0986683506 * d2_b1_mmdt + 0.101873458 * d2_b2_mmdt + 0.0986683506 * d2_a1_b1_mmdt +
↪  0.331194728 * m2_b1_mmdt + 0.0539714566 * n2_b1_mmdt + 3.24454093 * mass_mmdt + 0.503474228 *
↪ multiplicity - 1.50898053

      N8 [path=LLR; heap=8; reachable]
        h_1_8(x) = 7.32169007e-19 * zlogz - 2.06778204e-19 * d2_b1_mmdt - 1.91895639e-18 * d2_a1_b1_mmdt
↪  + 4.16333634e-17 * mass_mmdt + 5.02329003e-19
        <= 0 -> L2; > 0 -> L3

        L2 [path=LLRL; heap=17; reachable]
          f_1_2(x) = 0.398116558 * d2_b1_mmdt + 0.398116558 * d2_a1_b1_mmdt - 0.660830844 *
↪ d2_a1_b2_mmdt - 0.114827421 * m2_b2_mmdt - 0.310979867 * n2_b2_mmdt + 0.846928396 * multiplicity -
↪ 0.9437156

        L3 [path=LLRR; heap=18; reachable]
          f_1_3(x) = 0.244294729 * zlogz + 0.880510729 * d2_b2_mmdt - 1.42081066 * m2_b1_mmdt +
↪ 1.71265082 * multiplicity - 2.14718376

    N4 [path=LR; heap=4; reachable]
      h_1_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT

      N9 [path=LRL; heap=9; reachable]
        h_1_9(x) = 0.128095519 * m2_b1_mmdt + 0.00063403479 * multiplicity - 0.0832689733
        <= 0 -> L4; > 0 -> L5

        L4 [path=LRLL; heap=19; reachable]
          f_1_4(x) = 0.232595756 * zlogz + 0.336338146 * c1_b0_mmdt + 0.186230458 * d2_b1_mmdt +
↪ 0.186230458 * d2_a1_b1_mmdt - 0.0975771333 * m2_b1_mmdt + 0.100432843 * m2_b2_mmdt - 0.712895383 *
↪ multiplicity - 1.51109101

        L5 [path=LRLR; heap=20; reachable]
          f_1_5(x) = 0.26497515 * d2_b2_mmdt + 0.322656527 * n2_b2_mmdt - 0.656358636

      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_1_10(x) = 3.11856008e-17 * zlogz + 1.03393415e-17 * d2_b1_mmdt + 3.62950285e-17 *
↪ d2_a1_b1_mmdt - 3.214346e-17 * m2_b1_mmdt - 5.71317478e-17 * m2_b2_mmdt + 5.83355278e-17 * mass_mmdt +
↪  1.70316265e-11 * multiplicity - 1.89719708e-11
        <= 0 -> L6; > 0 -> L7

        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_1_6(x) = 0

        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_1_7(x) = 0

  N2 [path=R; heap=2; reachable]
    h_1_2(x) = 1.08273826 * c1_b2_mmdt + 0.24935691 * c2_b2_mmdt - 0.0971598576 * d2_b1_mmdt +
↪ 0.256354852 * d2_b2_mmdt - 0.0929252735 * d2_a1_b1_mmdt - 0.228292085 * m2_b1_mmdt + 0.0567631958 *
↪ mass_mmdt + 0.28027011 * multiplicity - 0.13138567
    <= 0 -> N5; > 0 -> N6
```

```
    N5 [path=RL; heap=5; reachable]
      h_1_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_1_11(x) = -0.259890766 * d2_a1_b2_mmdt + 0.208572797 * m2_b1_mmdt + 3.31046124 * mass_mmdt -
↪ 0.0387863599
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_1_8(x) = 1.49591861 * zlogz + 0.441575961 * c1_b0_mmdt + 1.71623752 * c1_b2_mmdt +
↪ 0.707286758 * d2_a1_b2_mmdt + 0.136799401 * n2_b1_mmdt - 0.0922364294 * n2_b2_mmdt - 4.1902612 *
↪ mass_mmdt + 0.994771372 * multiplicity + 0.902352018
        L9 [path=RLLR; heap=24; reachable]
          f_1_9(x) = 1.06616852 * zlogz + 0.26139261 * c1_b0_mmdt - 1.59073627 * c1_b1_mmdt - 3.31259981
↪  * c2_b1_mmdt + 2.90835606 * c2_b2_mmdt + 0.0402529973 * d2_b1_mmdt + 0.0402529973 * d2_a1_b1_mmdt +
↪ 3.30625 * d2_a1_b2_mmdt - 2.78491928 * m2_b1_mmdt + 0.342727361 * m2_b2_mmdt - 0.69660259 *
↪ n2_b2_mmdt + 1.00497391 * mass_mmdt + 2.54220983 * multiplicity + 0.0829818883
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_1_12(x) = 3.32650029e-05 * c1_b2_mmdt + 1.11948505e-05 * c2_b2_mmdt + 2.49204468e-05 *
↪ d2_a1_b2_mmdt + 1.99509567e-06 * m2_b2_mmdt + 0.000137262248 * n2_b1_mmdt + 9.50083532e-05
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_1_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_1_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_1_6(x) = 4.94065646e-324
      <= 0 -> N13; > 0 -> N14; CONSTANT: always RIGHT
      N13 [path=RRL; heap=13; UNREACHABLE (constant ancestor)]
        h_1_13(x) = -2.68948213e-10 * zlogz - 5.82051432e-11 * c2_b1_mmdt - 2.85373406e-10 * d2_b1_mmdt +
↪  2.05721037e-10 * d2_b2_mmdt - 1.06470833e-10 * d2_a1_b1_mmdt + 9.05405039e-12 * d2_a1_b2_mmdt +
↪ 3.42554664e-10 * m2_b2_mmdt + 2.13357549e-10 * n2_b2_mmdt - 8.89638165e-10 * mass_mmdt + 1.4924407e
↪ -09 * multiplicity - 1.72964941e-09
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_1_12(x) = 0
        L13 [path=RRLR; heap=28; UNREACHABLE (constant ancestor)]
          f_1_13(x) = 0
      N14 [path=RRR; heap=14; reachable]
        h_1_14(x) = -0.0357752902 * c1_b2_mmdt + 1.00738533 * mass_mmdt - 1.85526559
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; reachable]
          f_1_14(x) = 0.992368325 * zlogz - 4.80052499 * c1_b1_mmdt + 2.90036981 * c1_b2_mmdt -
↪ 0.714839331 * c2_b1_mmdt - 0.349819173 * d2_b1_mmdt - 0.0440891415 * d2_b2_mmdt - 0.349819173 *
↪ d2_a1_b1_mmdt + 0.777214702 * d2_a1_b2_mmdt + 0.176693713 * m2_b1_mmdt + 2.01859804 * n2_b1_mmdt -
↪ 1.16458901 * n2_b2_mmdt - 1.53336443 * mass_mmdt + 0.283339646 * multiplicity + 2.63807278
        L15 [path=RRRR; heap=30; reachable]
          f_1_15(x) = -0.485616316 * c1_b0_mmdt - 0.625011056 * c2_b2_mmdt + 0.51939675 * d2_b1_mmdt +
↪ 0.199901858 * d2_b2_mmdt + 0.51939675 * d2_a1_b1_mmdt - 0.491209214 * m2_b2_mmdt + 2.38441955 *
↪ mass_mmdt - 0.280424752 * multiplicity - 5.0660948
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Implementation and reproducibility

Tree 2 (scientific tree $t = 3$).

```
TREE 2: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).
N0 [path=root; heap=0; reachable]
  h_2_0(x) = 0.451963069 * c1_b2_mmdt - 0.00922949319 * m2_b1_mmdt + 0.315856319 * mass_mmdt +
↪ 0.0990362496
  <= 0 -> N1; > 0 -> N2
  N1 [path=L; heap=1; reachable]
    h_2_1(x) = -0.0393006101 * zlogz + 0.0190956747 * multiplicity + 0.00401865025
    <= 0 -> N3; > 0 -> N4
```

```
    N3 [path=LL; heap=3; reachable]
      h_2_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT
      N7 [path=LLL; heap=7; reachable]
        h_2_7(x) = -0.338370941 * zlogz - 0.197456658 * c1_b0_mmdt + 0.505337081 * multiplicity +
↪ 0.323631119
        <= 0 -> L0; > 0 -> L1
        L0 [path=LLLL; heap=15; reachable]
          f_2_0(x) = 0.391007555 * c1_b0_mmdt + 2.7854501 * c1_b1_mmdt - 5.98969282 * c1_b2_mmdt +
↪ 0.733907917 * c2_b1_mmdt - 0.0568167023 * d2_b2_mmdt - 0.461051796 * m2_b1_mmdt + 0.358625148 *
↪ m2_b2_mmdt + 0.00378192334 * n2_b2_mmdt - 2.04581277 * mass_mmdt - 0.432532063 * multiplicity -
↪ 1.93961875
        L1 [path=LLLR; heap=16; reachable]
          f_2_1(x) = -0.441179138 * zlogz - 3.22570484 * c1_b2_mmdt - 0.0681594455 * d2_b1_mmdt -
↪ 0.0681594455 * d2_a1_b1_mmdt + 0.311882288 * d2_a1_b2_mmdt - 0.131522611 * n2_b1_mmdt - 0.234805732 *
↪  mass_mmdt + 0.648265323 * multiplicity - 0.763080965
      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_2_8(x) = 0.00394358152 * mass_mmdt - 0.000176803622
        <= 0 -> L2; > 0 -> L3
        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_2_2(x) = 0
        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_2_3(x) = 0
    N4 [path=LR; heap=4; reachable]
      h_2_4(x) = 0.748689614 * c1_b0_mmdt + 0.669457034 * c2_b1_mmdt + 0.101051343
      <= 0 -> N9; > 0 -> N10
      N9 [path=LRL; heap=9; reachable]
        h_2_9(x) = 0
        <= 0 -> L4; > 0 -> L5; CONSTANT: always LEFT
        L4 [path=LRLL; heap=19; reachable]
          f_2_4(x) = -0.389784207 * zlogz - 0.209597106 * c1_b0_mmdt - 3.08379498 * c1_b1_mmdt -
↪ 1.4593788 * c2_b1_mmdt + 0.0255711958 * d2_b1_mmdt - 0.0574990566 * d2_b2_mmdt + 0.0255711958 *
↪ d2_a1_b1_mmdt + 0.973389446 * d2_a1_b2_mmdt + 0.250244728 * m2_b2_mmdt - 0.782019914 * n2_b1_mmdt +
↪ 0.0194565833 * n2_b2_mmdt + 2.61367386 * mass_mmdt + 0.993769728 * multiplicity + 0.228651881
        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_2_5(x) = 0
      N10 [path=LRR; heap=10; reachable]
        h_2_10(x) = -0.615381338 * zlogz - 0.780626703 * multiplicity - 0.248174582
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; reachable]
          f_2_6(x) = 0.405855831 * zlogz + 1.19454522 * c1_b0_mmdt - 0.171973875 * c1_b1_mmdt -
↪ 3.01430074 * c2_b1_mmdt + 1.66035781 * c2_b2_mmdt + 0.195722301 * d2_b2_mmdt - 0.470821689 *
↪ d2_a1_b2_mmdt + 0.725279387 * m2_b1_mmdt - 0.0949253938 * m2_b2_mmdt + 1.53492 * n2_b1_mmdt -
↪ 0.872843694 * n2_b2_mmdt + 1.27018591 * mass_mmdt + 0.644659641 * multiplicity + 0.542419575
        L7 [path=LRRR; heap=22; reachable]
          f_2_7(x) = 3.48806978 * zlogz + 7.81025712 * c1_b0_mmdt - 0.95346229 * c1_b2_mmdt +
↪ 0.280675132 * c2_b1_mmdt + 0.0334405957 * d2_b2_mmdt - 1.01384781 * d2_a1_b2_mmdt + 1.85257453 *
↪ m2_b1_mmdt - 0.533779879 * m2_b2_mmdt - 0.647558288 * n2_b1_mmdt + 0.591781799 * n2_b2_mmdt -
↪ 1.08156364 * mass_mmdt + 1.20551885 * multiplicity - 2.42799078
  N2 [path=R; heap=2; reachable]
    h_2_2(x) = 0.168562252 * c2_b2_mmdt - 0.0982298021 * d2_a1_b2_mmdt + 0.37090396 * m2_b1_mmdt +
↪ 0.113712418 * multiplicity - 0.745275257
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_2_5(x) = 0.814138721 * mass_mmdt + 0.0531775224 * multiplicity - 0.29709854
      <= 0 -> N11; > 0 -> N12
      N11 [path=RLL; heap=11; reachable]
        h_2_11(x) = 4.94065646e-324
        <= 0 -> L8; > 0 -> L9; CONSTANT: always RIGHT
        L8 [path=RLLL; heap=23; UNREACHABLE (constant ancestor)]
          f_2_8(x) = 0
        L9 [path=RLLR; heap=24; reachable]
          f_2_9(x) = -0.273060581 * zlogz + 0.214560559 * c1_b0_mmdt + 2.79082823 * c1_b2_mmdt +
↪ 0.0872642558 * d2_b1_mmdt + 0.0872642558 * d2_a1_b1_mmdt + 0.0668304306 * d2_a1_b2_mmdt + 0.700471572
↪  * m2_b2_mmdt - 0.377933505 * n2_b2_mmdt + 0.446788216 * multiplicity + 0.784109979
```

```
      N12 [path=RLR; heap=12; reachable]
        h_2_12(x) = 2.06493224e-10 * c1_b2_mmdt + 6.8828261e-11 * c2_b1_mmdt + 2.89598635e-11 *
↪ m2_b1_mmdt + 5.35347149e-11 * m2_b2_mmdt - 1.75510092e-10
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; reachable]
          f_2_10(x) = 0.0172917486 * zlogz + 0.479280898 * c1_b0_mmdt + 0.895727863 * c1_b1_mmdt -
↪ 2.47138391 * c1_b2_mmdt + 1.46797745 * c2_b1_mmdt + 0.0674431371 * d2_b1_mmdt - 0.20536907 *
↪ d2_b2_mmdt + 0.0674431371 * d2_a1_b1_mmdt + 0.960087986 * m2_b2_mmdt - 0.399753552 * n2_b1_mmdt -
↪ 0.688302486 * n2_b2_mmdt + 5.79450008 * mass_mmdt + 0.0327603624 * multiplicity - 0.350562803
        L11 [path=RLRR; heap=26; reachable]
          f_2_11(x) = 0.851042201 * c1_b0_mmdt + 1.12087708 * c1_b2_mmdt - 2.14848846 * c2_b1_mmdt +
↪ 0.740207807 * d2_b1_mmdt + 0.740207807 * d2_a1_b1_mmdt + 1.30902686 * m2_b1_mmdt + 4.77523926 *
↪ mass_mmdt + 1.62965024
    N6 [path=RR; heap=6; reachable]
      h_2_6(x) = 0
      <= 0 -> N13; > 0 -> N14; CONSTANT: always LEFT
      N13 [path=RRL; heap=13; reachable]
        h_2_13(x) = 4.94065646e-324
        <= 0 -> L12; > 0 -> L13; CONSTANT: always RIGHT
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_2_12(x) = 0
        L13 [path=RRLR; heap=28; reachable]
          f_2_13(x) = -0.148614563 * zlogz + 1.11851788 * c1_b0_mmdt + 0.96743728 * c1_b1_mmdt +
↪ 0.00568823559 * c2_b2_mmdt + 0.40419135 * d2_b2_mmdt - 1.01497415 * d2_a1_b2_mmdt + 1.11598924 *
↪ m2_b1_mmdt + 1.21950507 * n2_b1_mmdt - 0.144073893 * n2_b2_mmdt + 4.08231683 * mass_mmdt -
↪ 0.113704284 * multiplicity - 0.734827284
      N14 [path=RRR; heap=14; UNREACHABLE (constant ancestor)]
        h_2_14(x) = 0.078412842 * c1_b2_mmdt - 0.128921279
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; UNREACHABLE (constant ancestor)]
          f_2_14(x) = 0
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_2_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Final comparison

Table 8: Ten-model validation comparison. Layout specifies tree count and maximum depth; D4 means depth four. P_{nz} counts learned nonzero scalar values. ϵ_s is interpolated at 1% background; $\epsilon_s^{\mathrm{strict}}$ uses a strict threshold accepting at most 525 background events. C++ times are measured with the same sparse reference kernel. M_{path} is a selected-path upper bound and M_{all} counts all structurally reachable affine functions, both after exact-zero and ±1 factor elimination.

Family	Layout	P_{nz}	ϵ_s [%]	$\epsilon_s^{\mathrm{strict}}$ [%]	AUC	C++ [μs/event]	M_{path}	M_{all}
XGBoost	100×D4	3099	55.48	55.48	0.92056	3.675	0	0
Axis / linear	3×D4	479	56.32	56.32	0.92576	0.101	35	386
Axis / linear	1×D4	228	54.74	54.74	0.92387	0.048	16	197
Axis / linear	1×D3	132	50.10	50.10	0.92020	0.049	16	117
Oblique / constant	5×D4	1250	55.40	54.75	0.92610	0.965	272	1088
Oblique / constant	3×D4	753	53.54	53.30	0.92318	0.542	160	656
Oblique / constant	1×D4	256	45.36	45.33	0.91247	0.171	64	224
Oblique / linear	5×D4	1275	57.73	57.73	0.93090	0.412	212	1096
Oblique / linear	3×D4	837	57.12	57.12	0.92967	0.320	160	734
Oblique / linear[c]	3×D4	418	56.20	56.20	0.92733	0.167	78	325

[c]Compact refit. Axis rows additionally use 45, 15 and 7 feature indices, respectively, and one direction bit per nonconstant cut. Their fixed unit routing coefficients are excluded from P_{nz}. XGBoost stores 1499 feature indices; sparse oblique storage can also require coordinate indices. These scalar counts are not complete memory-footprint estimates.

All selected TAO snapshots come from converged source stages; selection can choose an earlier checkpoint. The axis 3×D4 entry uses its regularized source: its unpenalized leaf refit was held for separation. The 837-coefficient oblique/linear entry uses a converged leaf-only stage; its held full stage is excluded. Representatives cover different scalar budgets within each family.

Timing: one CPU thread, 132,800 events, median of nine repeats after warmup; parsing, input conversion and I/O excluded. Native optimized XGBoost separately measured 1.478 μs/event, including probability output. Working points are calibrated to the stated reference arithmetic and must be rechecked after a change of reduction order or precision. The common C++ column is not a ranking of maximally optimized software implementations.

Multiplication counts exclude preprocessing, sigmoid, additions and comparisons; no shift or general constant-multiplier optimization is assumed beyond exact-zero and unit factors. They are arithmetic workloads, not synthesized DSP/LUT/BRAM use or FPGA latency. Results use the fixed validation split and do not include a new test evaluation.